daVinci-LLM: Towards the Science of Pretraining

Yiwei Qin[*1,2,3] Yixiu Liu[*1,2,3] Tiantian Mi[*1,3] Muhang Xie[*1,3] Zhen Huang[*1,3]
Weiye Si[1,2,3] Pengrui Lu[1,2,3] Siyuan Feng[1] Xia Wu[1] Liming Liu[1]
Jinlong Hou[1] Qipeng Guo[1] Yu Qiao[1] Pengfei Liu[†1,2,3]

[1]SII [2]SJTU [3]GAIR

daVinci-LLM daVinci-LLM Model daVinci-LLM Data

Abstract

The foundational pretraining phase determines a model's capability ceiling, as post-training struggles to overcome capability foundations established during pretraining, yet it remains critically under-explored. This stems from a structural paradox: organizations with computational resources operate under commercial pressures that inhibit transparent disclosure, while academic institutions possess research freedom but lack pretraining-scale computational resources. daVinci-LLM occupies this unexplored intersection, combining industrial-scale resources with full research freedom to advance the science of pretraining. We adopt a fully-open paradigm that treats openness as scientific methodology, releasing complete data processing pipelines, full training processes, and systematic exploration results. Recognizing that the field lacks systematic methodology for data processing, we employ the Data Darwinism framework—a principled L0-L9 taxonomy from filtering to synthesis. We train a 3B-parameter model from random initialization across 8T tokens using a two-stage adaptive curriculum that progressively shifts from foundational capabilities to reasoning-intensive enhancement. Through 200+ controlled ablations, we establish that: processing depth systematically enhances capabilities, establishing it as a critical dimension alongside volume scaling; different domains exhibit distinct saturation dynamics, necessitating adaptive strategies from proportion adjustments to format shifts; compositional balance enables targeted intensification while preventing performance collapse; how evaluation protocol choices shape our understanding of pretraining progress. By releasing the complete exploration process, we enable the community to build upon our findings and systematic methodologies to form accumulative scientific knowledge in pretraining.

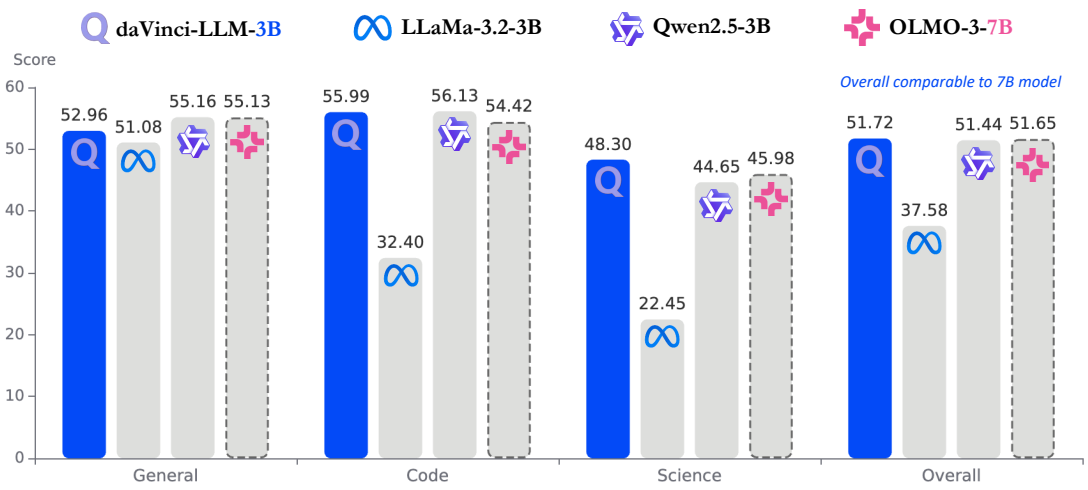


Figure 1: Performance comparison of daVinci-LLM-3B against baseline models with score across three capability domains, and overall score comparable to OLMo-3-7B.

* Equal contribution.
† Corresponding author.

Contents

1 Introduction . . . **3**

2 Data: What We Used and How We Processed It . . . **5**

2.1 Data Darwinism Framework . . . 5

2.2 Data Pool . . . 6

2.2.1 General . . . 7

2.2.2 Code . . . 7

2.2.3 Science . . . 8

2.2.4 QA . . . 10

3 Training Recipe: How We Trained It . . . **12**

3.1 Model Architecture . . . 12

3.2 Evaluation Protocol . . . 12

3.3 Training Methodology . . . 13

3.3.1 Stage 1: General Foundation Pretraining . . . 14

3.3.2 Stage-2: Reasoning Capability Enhancement . . . 15

3.4 Final Results . . . 16

4 Exploration: Why We Trained It This Way . . . **17**

4.1 Data Processing Depth: From Filtering to Synthesis . . . 18

4.1.1 Code Data Filtering: L3 Model-Based Filtering . . . 18

4.1.2 Math Data Quality: L4 Generative Refinement . . . 19

4.1.3 L5 Synthetic QA: Cognitive Completion . . . 19

4.2 Training Dynamics: Adaptive Data Strategies . . . 20

4.2.1 Domain Proportion Adjustment . . . 20

4.2.2 From Domain Adjustment to QA Introduction . . . 21

4.3 Data Mixture Design: Balancing and Intensifying . . . 22

4.3.1 Domain Balance: Code and Science Composition . . . 23

4.3.2 QA Concentration: Progressive Intensification . . . 23

4.4 Evaluation Validity: PPL-based vs. Generative-based . . . 24

5 Conclusion . . . **25**

A Evaluation Details . . . **31**

A.1 Benchmark Descriptions . . . 31

A.1.1 General Knowledge and Reasoning Benchmarks . . . 31

A.1.2 Code Generation Benchmarks . . . 32

A.1.3 Mathematics and STEM Reasoning Benchmarks . . . 32

B Training Implementation Decisions: LR Decay and QA Masking . . . **33**

C Prompts for Dataset Construction . . . **34**

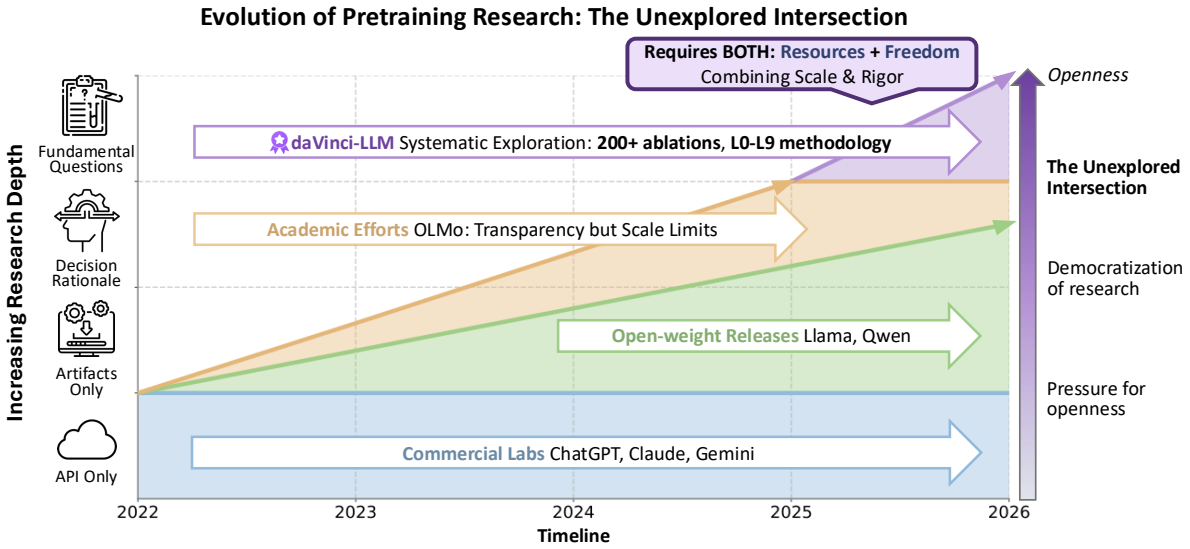


Figure 2: Evolution of pretraining research depth across institutional structures. The y-axis represents research depth from surface-level artifacts (API-only access) to fundamental scientific questions. The x-axis shows the temporal progression from 2022 to 2026. Commercial entities (blue) possess computational resources but remain constrained to API-level access due to competitive pressures. Open-weight releases (green, e.g., Llama, Qwen) provide model artifacts but withhold design rationale and negative results. Academic efforts (orange, e.g., OLMo) achieve transparency and research freedom but face severe scale limitations—making systematic exploration with 200+ configurations structurally infeasible. The top tier remains largely unexplored, as it requires the rare alignment of large-scale computational resources with the research freedom to publish comprehensive findings. DAVINCI-LLM (purple) occupies this intersection, conducting the extensive ablations and systematic disclosures necessary to advance the science of pretraining. By releasing the complete decision-making logic alongside the model weights, we bridge the structural gap between industrial scale and scientific transparency.

1 Introduction

The large language model ecosystem has evolved into a stratified landscape characterized by varying levels of transparency. At the most opaque tier lie closed-source commercial models (GPT [1, 2], Claude [3], Gemini [4, 5]) accessible only through APIs. The intermediate tier comprises open-weight models (LLaMA [6], Qwen [7, 8, 9, 10], DeepSeek [11]) that release checkpoints but withhold critical pretraining details—data compositions, mixture ratios, and training dynamics remain largely undisclosed. At the foundation are fully-open efforts that release complete training specifications. Despite this stratification, the field remains dominated by the first two paradigms. Yet as the ATOM (American Truly Open Models) Project [12] emphasizes, open language models are crucial for long-term competition by enabling the broader research community to pursue long-horizon transformative innovations rather than only immediate deployment priorities. However, research attention has concentrated disproportionately on accessible post-training techniques [13, 14, 15, 11, 16, 17, 18, 19, 20, 21, 22, 23], while the foundational pretraining phase, which determines a model's capability ceiling, remains critically under-explored.

This imbalance stems from structural constraints that create a fundamental paradox. Organizations with computational resources for large-scale pretraining operate under commercial pressures that favor rapid deployment over systematic exploration and inhibit transparent disclosure of training processes. Academic institutions possess research freedom but lack pretraining-scale infrastructure—even well-funded efforts like OLMo [24, 25] face severe scale limitations that make large-scale systematic exploration structurally infeasible, while confronting persistent challenges in sustaining both computational resources and key research personnel. The consequence is stark: precisely when emerging evidence demonstrates that pretraining choices fundamentally shape downstream capabilities [26], the community has limited ability to systematically investigate the principles governing how models acquire and organize knowledge during pretraining. Post-training techniques can refine and align model behavior, but struggle to fundamentally overcome the capability foundations established during pretraining—research [27, 28] shows that pretraining advantages are amplified rather than compensated for in subsequent training

Table 1: Transparency comparison across state-of-the-art LLMs. Unlike existing models, daVinci provides the complete scientific process, enabling the systematic investigation of pretraining dynamics. Symbols: ✔ fully open, ◎ patial disclosure or not released, ✖ not disclosed.

	Dimension	Llama 3	Qwen 3	YuLan	OLMo 3	daVinci
Model Artifacts	Model Weights	✔	✔	✔	✔	✔
	Training Code	✖	✖	✔	✔	✔
	Training Logs	✖	✖	✖	✔	✔
	Intermediate Checkpoints	✖	✖	✖	✔	✔
Data Openness	Data Composition	◎	◎	✔	✔	✔
	Processing Pipeline	◎	◎	✔	✔	✔
	Full Training Data	✖	✖	◎	✔	✔
	Processing Methodology	✖	✖	✖	✖	✔ *(L0-9)*
Scientific Process	Pretraining Ablations	✔	✖	✔	✔	✔
	Mixture Rationale	✖	✖	✖	◎	✔
	Decision Transparency	✖	✖	✖	◎	✔
	Negative Results	✖	✖	✖	✖	✔

phases.

We are positioned to address this gap by combining computational resources for billion-parameter training with the research freedom to investigate fundamental questions and publish comprehensive findings. Towards the science of pretraining, we adopt a fully-open paradigm (as shown in Table 1) that treats openness itself as scientific methodology, releasing not only model weights but complete training trajectories, data specifications, and ablation results documenting both what works and what fails. Our work is structured around three pillars, each contributing to transparency and reproducibility.

Data (Section 2) Recognizing that data quality fundamentally determines outcomes yet the field lacks systematic methodology for processing decisions, we adopt the Data Darwinism framework [29], a principled L0-L9 taxonomy organizing operations from basic filtering to content transformation to knowledge synthesis. We release our complete data processing pipeline and the processed datasets themselves. Our data pool spans general web text, code, science, and QA domains, with each source explicitly annotated by its Darwin Level to systematically organize our processing decisions and identify where further quality enhancement is feasible.

Training Recipe (Section 3) We train a 3B-parameter model from random initialization across 8T tokens using a two-stage curriculum: Stage 1 (6T tokens) establishes broad foundational capabilities through diverse web-scale corpora with progressive data adjustment, while Stage 2 (2T tokens) shifts toward reasoning-intensive enhancement by introducing large-scale structured QA data alongside continued exposure to code and science domains. We release all intermediate checkpoints at 5k-step intervals, complete hyperparameters, training logs, and the evolution of data mixture compositions across stages, providing a complete developmental trajectory from initialization to final model.

Exploration (Section 4) Towards the science of pretraining, we transform design decisions into systematically verifiable research questions. Through 200+ controlled ablations, we systematically investigate key questions spanning four thematic areas about pretraining: (1) **Data Processing Depth**: How does hierarchical data processing systematically enhance model capabilities? (2) **Training Dynamics**: How should data strategies and curriculum adapt during training? (3) **Data Mixture Design**: How to balance targeted enhancement with general capability preservation? (4) **Evaluation Validity**: Which evaluation protocols reliably measure base model pretraining progress? Through these investigations, we find that processing depth systematically enhances capabilities, different domains exhibit distinct training dynamics requiring adaptive strategies, and mixture design critically determines the balance between capability enhancement and preservation. We document both successful configurations and failed experiments, providing empirical evidence to inform each dimension of pretraining decisions.

Our contributions span three dimensions: (1) **Complete research materials**: We release model weights, intermediate checkpoints, and processed datasets, enabling researchers to analyze capability emergence, reproduce training processes, and conduct extended investigations. (2) **Question-driven pretraining science**: By transforming key pretraining decisions into systematically verifiable research questions, we provide empirical understanding of data quality, training dynamics, and mixture strategies, offering reference points for researchers facing similar decisions. (3) **Transferable methodological**

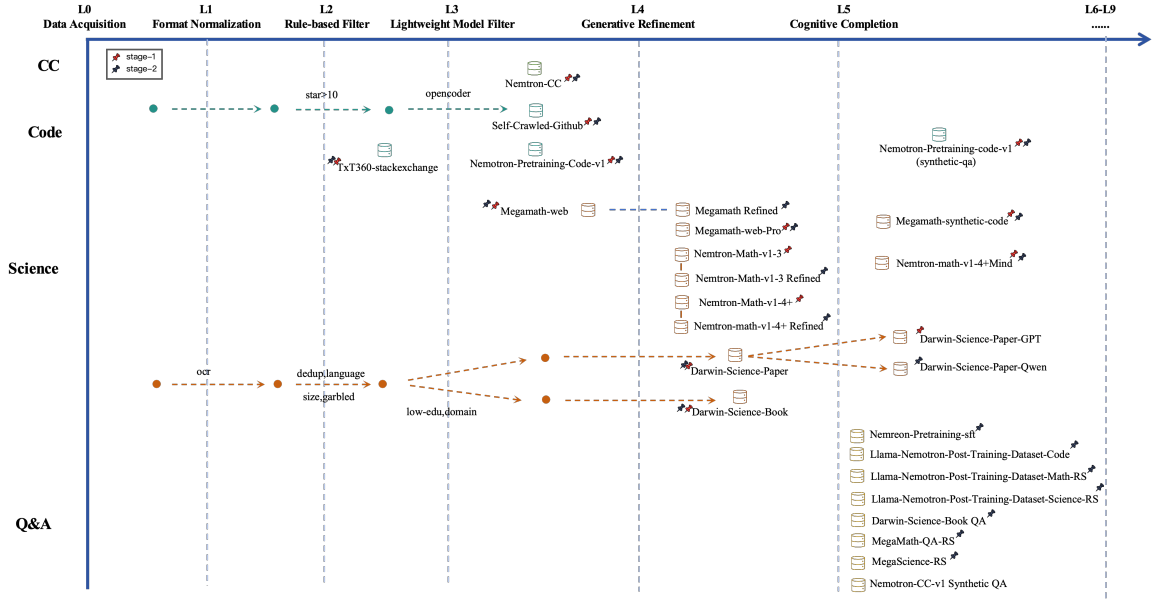


Figure 3: Mapping of our pretraining data sources onto the Data Darwinism L0–L9 taxonomy across different training stage.

foundations: The Data Darwinism framework, systematic exploration methods, and complete documentation of successes and failures constitute reusable research infrastructure, enabling the community to build upon documented boundary conditions and form accumulative scientific knowledge in pretraining.

To facilitate reproducibility and support the research community, we publicly release all datasets produced through our own processing and synthesis pipelines, together with the complete curation toolkit comprising all prompts and processing code. We further release all intermediate and final model checkpoints saved throughout training, training logs, ablation results, and our full evaluation suite. We hope these releases can help the community better understand, reproduce, and build upon our work.

2 Data: What We Used and How We Processed It

Data transparency remains one of the most underserved dimensions of open pretraining research. While open-weight releases have made model checkpoints accessible, the data decisions underlying those models, including what sources were selected and how they were processed, remain largely opaque. We address this directly: beyond releasing our complete training corpus, we document the processing depth of every data source through the Data Darwinism framework (L0–L9) [29], making our curation decisions explicit and traceable. Our pretraining corpus is organized into four categories of General, Code, Science, and QA, drawn from a combination of existing open-source datasets and data collected directly from public repositories, with each source annotated by its corresponding Darwin Level and further processed where meaningful quality gains can be achieved.

2.1 Data Darwinism Framework

The quality of pretraining data is fundamentally shaped by how it has been collected, filtered, and processed. Yet the field currently lacks a systematic framework for categorizing and comparing these operations, making it difficult to reason about quality differences across heterogeneous data sources. Data Darwinism was proposed precisely to address this gap, organizing data processing operations into a principled ten-level taxonomy (L0–L9). We adopt this framework to assess the processing depth of every data source in our training corpus—not only to understand where each dataset currently stands in the hierarchy, but also to identify whether further processing is feasible and whether the potential quality gains justify the additional investment.

Underlying this hierarchy is a coherent evolutionary logic: data processing begins with the selection and preservation of existing content, progressively moves toward active rewriting and enrichment, and ultimately reaches the capacity to synthesize entirely new content from scratch. In parallel, the agents driving these operations shift from hand-crafted deterministic rules, to lightweight classification models, to frontier large language models capable of reasoning and generation. We now describe each level in turn:

L0: Data Acquisition. At this foundational stage, raw data is gathered from diverse sources including web crawls, PDF repositories, code platforms, and curated databases. The collected data exists in highly variable formats such as HTML, PDF, and binary files, and typically contains significant noise and duplication. The primary challenges at this level lie in achieving broad coverage, maintaining data provenance, and managing large-scale storage infrastructure.

L1: Format Normalization. At this stage, heterogeneous raw data is converted into unified, training-ready text representations. For document-based sources, key operations include OCR processing of scanned PDFs and HTML parsing to extract clean content. No content is filtered here; the goal is uniform processability while preserving structural fidelity across sources.

L2: Rule-based Filtering. This is the first stage of quality control, where deterministic pattern-based rules are applied to remove objectively identifiable problematic content: near-duplicates detected via MinHash LSH, excessively short or malformed text, non-target languages, and garbled text from encoding errors. The approach requires no learned models, runs efficiently on CPU infrastructure, and achieves substantial volume reduction while remaining fully interpretable.

L3: Lightweight Model Filtering. Unlike L2, which operates on surface patterns, this stage introduces semantic-level quality assessment using pretrained lightweight classifiers. Tasks such as educational value scoring, domain identification, and document type classification enable more nuanced filtering decisions than rules alone can support. Importantly, this remains a pure selection stage: documents are retained or discarded based on predicted quality, but their content is never modified.

L4: Generative Refinement. This stage marks a qualitative shift from selection to active, model-driven transformation. Medium-to-large generative models are deployed to purify content by removing structural noise such as navigation elements, reference lists, OCR artifacts, and formatting defects, as well as repairing fragmented text, while strictly adhering to the original content. A critical constraint is that this stage must act as a faithful refiner: no external knowledge may be introduced, and the output must remain semantically equivalent to the input.

L5: Cognitive Completion. At this stage, frontier LLMs enrich data by making implicit reasoning explicit. Research and technical documents are typically written for expert audiences, characterized by compressed logical steps, assumed background knowledge, and implicit derivations that create a learnability gap for language models. This stage bridges that gap through reasoning reconstruction, terminological explication, and pedagogical bridging, producing content that retains full scientific fidelity while substantially lowering the cognitive barrier for model internalization.

L6–L9: Higher-Order Synthesis. The upper levels of the hierarchy address increasingly ambitious forms of data generation. Contextual Completion (L6) expands documents by integrating external references and background knowledge to create self-contained artifacts. Environment Synthesis (L7) constructs executable environments in which data objects can be validated through actual execution. Ecosystem Synthesis (L8) builds dynamic multi-agent systems where diverse intelligent entities interact and generate emergent data through sustained collaboration. World Synthesis (L9) represents the theoretical apex of the framework, aspiring to construct comprehensive simulated worlds as a source of essentially unlimited synthetic training data.

It is worth noting that these levels are not mutually exclusive one-time passes: any operation at a given level can be applied multiple times, with different models, prompts, or parameters, each targeting different aspects of quality to achieve progressively deeper processing. Furthermore, the ordering of operations need not strictly follow the level hierarchy—for instance, a dataset may undergo L4 Generative Refinement before being subjected to L3 model-based filtering, if such an ordering better suits the characteristics of the source data.

Throughout this work, we annotate each data source with its corresponding Darwin Level to make our curation decisions explicit and transparent. For a subset of sources, we further apply higher-level processing operations to actively improve their quality. This allows us to reason systematically about the depth of processing across our entire corpus, and to identify where additional effort is most likely to yield meaningful gains.

2.2 Data Pool

We curate our pretraining corpus from five major categories: CC, Code, Math, Science, and QA. For each data source, we annotate its corresponding Darwin Level to make our curation decisions explicit and transparent. Table 2 summarizes the composition and scale of the full data pool across training stages, and Figure 3 provides an overview of the Darwin Level assigned to each data source.

Table 2: Data pool composition and token allocation across training stages. Shaded category rows show each category's total pool size and the tokens allocated per stage; data source rows indicate whether the source is used (✓) or not used (–) in each stage.

Data Source	Pool Size	Level	Stage 1		Stage 2	
			Stage 1-1	Stage 1-2	Stage 2-1	Stage 2-2
General	**4.28T**		**2.73T** *(68.2%)*	**1.11T** *(55.42%)*	**100B** *(10%)*	**188.4B** *(18.84%)*
Nemotron-CC-v1	4.28T	L3	✓	✓	✓	✓
Code	**598B**		**381B** *(9.53%)*	**233B** *(11.66%)*	**300** *(30%)*	**26.1B** *(2.61%)*
Self-Crawled GitHub	187B	L3	✓	✓	✓	✓
Nemotron-Pretraining-Code-v1-non-synthetic	220B	L3	✓	✓	✓	✓
Nemotron-Pretraining-Code-v1-synthetic-code	171B	L5	✓	✓	✓	✓
TxT360-Stack-Exchange	20B	L2	✓	✓	✓	✓
Science	**1.94T**		**891B** *(22.27%)*	**658B** *(32.92%)*	**300B** *(30%)*	**85.5B** *(8.85%)*
MegaMath-Web	231B	L3	✓	✓	–	–
MegaMath-Web-Pro	13B	L4	✓	✓	✓	✓
MegaMath Refined	176B	L4	–	–	✓	✓
MegaMath-Synth-Code	5B	L5	✓	✓	✓	✓
Nemotron-CC-Math-v1-3	81B	L4	✓	✓	–	–
Nemotron-CC-Math-v1-4+	52B	L4	✓	✓	–	–
Nemotron-CC-Math-v1-4+-MIND	74B	L5	✓	✓	✓	✓
Nemotron-CC-Math-v1-3 Refined	68B	L4	–	–	✓	✓
Nemotron-CC-Math-v1-4+ Refined	47B	L4	–	–	✓	✓
Darwin-Science-Book	251B	L4	✓	✓	✓	✓
Darwin-Science-Paper	215B	L4	✓	✓	✓	✓
Darwin-Science-Paper-GPT	290B	L5	✓	✓	–	–
Darwin-Science-Paper-Qwen	440B	L5	–	–	✓	✓
QA	**734B**		**0** *(0%)*	**0** *(0%)*	**300B** *(30%)*	**700B** *(70%)*
Nemotron-CC-v1 Synthetic QA	492B	L5	–	–	–	–
Nemotron-Pretraining-SFT-v1-Code	21B	L5	–	–	✓	✓
Nemotron-Pretraining-SFT-v1-Math	138B	L5	–	–	✓	✓
Nemotron-Pretraining-SFT-v1-General	12B	L5	–	–	✓	✓
Llama-Nemotron-Post-Training-Dataset-Code	5B	L5	–	–	✓	✓
Llama-Nemotron-Post-Training-Dataset-Math-RS	10B	L5	–	–	✓	✓
Llama-Nemotron-Post-Training-Dataset-Science-RS	0.4B	L5	–	–	✓	✓
MegaMath-QA-RS	9B	L5	–	–	✓	✓
MegaScience-RS	1B	L5	–	–	✓	✓
Darwin-Science-Book QA	46B	L5	–	–	✓	✓
Total	**~7.58T**		**4T**	**2T**	**1T**	**1T**

2.2.1 General

General web text forms the backbone of our pretraining corpus, providing broad coverage across topics, writing styles, and knowledge domains. Our general corpus is primarily drawn from Common Crawl, with quality filtering applied to balance coverage and data quality.

Nemotron-CC-v1. We adopt the non-synthetic portion of *Nemotron-CC-v1* [30] as our web-scale general corpus. *Nemotron-CC-v1* is built from 99 snapshots of Common Crawl and undergoes text extraction, English language filtering, and global deduplication, followed by an ensemble of classifiers combining educational value and informativeness signals that scores each document and groups them into five quality tiers based on downstream task performance—enabling precise control over the quality–diversity trade-off across training stages. This complete processing pipeline places *Nemotron-CC-v1* at Darwin Level L3, contributing approximately 4.28T tokens to our training corpus.

2.2.2 Code

Code data equips the model with structured, executable knowledge and exposes it to formal reasoning patterns grounded in programming languages. Our code corpus is assembled from a combination of self-crawled GitHub repositories and existing open-source code datasets, spanning both real-world source code and synthetically generated coding examples.

Self-Crawled GitHub. We directly crawled public GitHub repositories and applied a minimum threshold of 10 stars per repository as an initial rule-based quality gate, ensuring that only repositories

with demonstrated community adoption are retained. The collected source files were then organized and passed through OpenCoder's filtering pipeline [31], which removes low-quality or non-informative code files through lightweight model-based assessment, bringing the dataset to Darwin Level L3. This process yields approximately 187B tokens.

Nemotron-Pretraining-Code-v1. We incorporate two complementary subsets from *Nemotron-Pretraining-Code-v1* [32], after cross-deduplication with our self-crawled GitHub collection.

- *Nemotron-Pretraining-Code-v1-non-synthetic.* This subset consists of real-world source code collected from GitHub. Repositories undergo license-based filtering to retain only permissively licensed code, followed by both exact and fuzzy deduplication to address the pervasive cross-repository redundancy characteristic of open-source ecosystems. The OpenCoder filtering pipeline is then applied to remove files that are low-quality or detrimental for LLM pretraining, bringing this subset to Darwin Level L3 and yielding approximately 220B tokens.
- *Nemotron-Pretraining-Code-v1-synthetic-code.* This subset is generated by prompting an LLM to produce question-answer pairs grounded in short code snippets, where the model is asked to both formulate and solve coding questions across 11 programming languages. The resulting natural language–code interleaved pairs are filtered post-hoc through language-specific heuristics such as Python AST parsing, reaching Darwin Level L5 and yielding approximately 171B tokens.

TxT360-Stack-Exchange. We incorporate *txt360-stack-exchange* [33] as a source of technical community discourse. This dataset is compiled from the Stack Exchange network, covering 364 sub-communities spanning programming, mathematics, science, and numerous other technical domains. Raw data is extracted from archived XML dumps, where posts and comments are parsed to reconstruct the full threaded discussion hierarchy, preserving the collaborative reasoning process characteristic of community-driven knowledge building. Rule-based cleaning and format normalization are applied to ensure consistent structure, placing the dataset at Darwin Level L2 and contributing approximately 20B tokens.

2.2.3 Science

Scientific knowledge forms an important part of a well-rounded pretraining corpus, enriching the model with structured, knowledge-dense content spanning a broad range of disciplines. Our scientific corpus is assembled from a combination of existing open-source datasets and documents collected directly from public repositories, with additional processing applied to improve quality where appropriate.

MegaMath. We incorporate three subsets from *MegaMath* [34] as part of our mathematical pretraining corpus. MegaMath is curated from diverse math-focused sources via a two-stage, coarse-to-fine web extraction pipeline over 99 Common Crawl snapshots, combined with math-related code recall from Stack-V2 and LLM-based synthetic data generation.

- *MegaMath-Web.* This subset consists of mathematical content extracted from Common Crawl using a two-stage pipeline: an initial fast extraction pass with Resiliparse followed by high-fidelity re-extraction with trafilatura on math-optimized HTML, with fastText-based math filtering and MinHash LSH deduplication applied throughout. The result is filtered through a fastText classifier trained on LLM-annotated math-relevance labels, reaching Darwin Level L3 and contributing approximately 231B tokens.
- *MegaMath-Web-Pro.* This subset is a high-quality subset of MegaMath-Web produced by first applying the FineMath classifier with a dynamic educational-value threshold (score ≥ 4 for older snapshots; score ≥ 3 for recent ones) to select high-quality documents, followed by LLM-driven refinement using Llama-3.3-70B-Instruct to remove noise and reorganize content into a logically structured, information-dense form while preserving the original length. This reaches Darwin Level L4 and contributes approximately 13B tokens.
- *MegaMath-Synth-Code.* This subset consists of LLM-generated natural language–code interleaved pairs targeting mathematical reasoning, produced by prompting LLMs to generate structured blocks of mathematical text, symbolic expressions, and executable Python code grounded in MegaMath-Web documents, with syntax and runtime verification via AST filtering and execution checks. It reaches Darwin Level L5 and contributes approximately 5B tokens.

MegaMath Refined. We apply L4 Generative Refinement to *MegaMath-Web* using Qwen3-235B-A22B-Instruct [9], following the same prompt design as MegaMath-Web-Pro. The refinement strategy prompts the model to extract key facts and concepts, remove noisy or irrelevant content, and reorganize the material into a logically structured, information-dense form while preserving the original length (see Appendix C for the full prompt). The refined version of MegaMath-Web reaches Darwin Level L4 and contributes approximately 176B tokens.

Nemotron-CC-Math-v1. We incorporate three subsets from *Nemotron-CC-Math-v1* [35], a high-quality mathematical corpus constructed from 98 Common Crawl snapshots (2014–2024) spanning over 980,000 unique domains. Its pipeline identifies math-relevant pages by aggregating URL lists from existing open math datasets, renders HTML via the Lynx text-based browser, applies a Phi-4-based cleanup pass to remove boilerplate and normalize heterogeneous mathematical representations into unified LaTeX format, and then uses the FineMath classifierto score each document on a 1–5 scale for quality filtering, followed by fuzzy deduplication and benchmark decontamination.

- *Nemotron-CC-Math-v1-3.* This subset retains documents with FineMath classifier scores of 3. The Phi-4-based cleanup pass applied during the original pipeline constitutes an L4 Generative Refinement operation, bringing this subset to Darwin Level L4 and contributing approximately 81B tokens.
- *Nemotron-CC-Math-v1-4+.* This subset retains only the highest-quality documents with FineMath classifier scores of 4–5. As with Nemotron-CC-Math-v1-3, the Phi-4-based cleanup pass places this subset at Darwin Level L4, contributing approximately 52B tokens.
- *Nemotron-CC-Math-v1-4+-MIND.* This subset is obtained by applying the MIND framework [36] to *Nemotron-CC-Math-v1-4+*, converting each mathematical document into structured multi-turn dialogues via diverse conversational prompt templates (e.g., Teacher-Student, Problem-Solving, Debate) that reconstruct implicit reasoning steps and lower the cognitive barrier for model internalization, while strictly preserving the original content without introducing external knowledge. It reaches Darwin Level L5 and contributes approximately 74B tokens.

Nemotron-CC-Math-v1 Refined. Building on the L4 Generative Refinement already applied during the original dataset construction, we apply a second round of L4 refinement to *Nemotron-CC-Math-v1-3* and *Nemotron-CC-Math-v1-4+* using the stronger Qwen3-235B-A22B-Instruct [9], with a more targeted prompt that instructs the model to extract key facts and concepts, remove noisy or irrelevant content, and reorganize the material into a logically structured, information-dense form while preserving the original length (see Appendix C for the full prompt). This reflects our broader treatment of Darwin Level operations as iterative rather than one-time passes.

- *Nemotron-CC-Math-v1-3 Refined.* The refined version of Nemotron-CC-Math-v1-3, reaching Darwin Level L4 and contributing approximately 68B tokens.
- *Nemotron-CC-Math-v1-4+ Refined.* The refined version of Nemotron-CC-Math-v1-4+, reaching Darwin Level L4 and contributing approximately 47B tokens.

Darwin-Science. We construct *Darwin-Science* as our primary scientific corpus, built from raw PDFs of scientific books and academic papers sourced from publicly accessible online repositories and open-source datasets including PubMed and arXiv. Both sources share a common L0–L3 processing pipeline. Raw PDFs are first converted into machine-readable text using olmOCR-7B-0225-preview [37], a vision-language model optimized for document text extraction. The resulting text then undergoes deduplication via MinHash LSH, followed by rule-based filtering that discards files below 8kb, documents with excessive garbled characters resulting from OCR errors, and non-English content. All retained documents are subsequently annotated using EAI-Distill-0.5B [38], a lightweight classifier that performs educational value scoring and field-of-discipline classification across nine major domains; documents with no educational value are filtered out. Finally, all documents are classified into book and paper categories—using metadata where available, and Qwen2.5-7B-Instruct [8] for ambiguous cases—as the two document types exhibit distinct learnability characteristics that warrant different downstream processing.

- *Darwin-Science-Book.* Scientific books are processed through L4 Generative Refinement using GPT-OSS-120B [39], which removes structural noise such as table of contents entries, reference lists, headers and footers, and OCR artifacts, while repairing formatting defects such as fragmented text and damaged formulas without altering the underlying content. The detailed prompt is provided in Appendix C. This process yields approximately 251 tokens.

- *Darwin-Science-Paper*. The same L4 Generative Refinement pipeline is applied to academic papers using GPT-OSS-120B, yielding approximately 215 tokens. These L4-processed papers serve as the foundation for further L5 processing.
- *Darwin-Science-Paper-GPT*. Building on the L4-processed papers, we apply L5 Cognitive Completion using GPT-OSS-120B to bridge the learnability gap inherent in expert-oriented scientific writing. The augmentation targets three dimensions: expanding implicit logical leaps into explicit step-by-step derivations (Reasoning Reconstruction), contextualizing domain-specific terminology within the narrative flow rather than assuming prior mastery (Terminological Explication), and grounding abstract concepts in concrete analogies and established knowledge (Pedagogical Bridging). The detailed prompt is provided in Appendix C. This yields approximately 290B tokens.
- *Darwin-Science-Paper-Qwen*. We also apply L5 Cognitive Completion directly to the L4-processed papers using Qwen3-235B-A22B-Instruct [9]. We hypothesize that a stronger model is better equipped to reconstruct implicit reasoning and produce richer pedagogical enrichment, potentially yielding greater learnability gains. This yields approximately 440 tokens.

2.2.4 QA

Question-answer pairs serve as a natural complement to raw text pretraining data, and our QA data covers three broad domains: code, general, and science. Sources are drawn from a mix of existing high-quality collections, rejection-sampled open-source post-training datasets by Qwen3-32B in non-thinking mode, and QA pairs synthesized directly from scientifc documents. All subsets reach Darwin Level L5.

Table 3: Domain coverage of QA data sources. Each source is categorized into one of three domains: General, Code, and Science (including Mathematics).

Data Source	General	Code	Science
Nemotron-CC-v1 Synthetic QA (492B)	✔		
Nemotron-Pretraining-SFT-v1-General (12B)	✔		
Nemotron-Pretraining-SFT-v1-Code (21B)		✔	
Llama-Nemotron-Post-Training-Code (5B)		✔	
Nemotron-Pretraining-SFT-v1-Math (138B)			✔
Llama-Nemotron-Post-Training-Math-RS (10B)			✔
Llama-Nemotron-Post-Training-Science-RS (0.4B)			✔
MegaMath-QA-RS (9B)			✔
MegaScience-RS (1B)			✔
Darwin-Science-Book QA (46B)			✔

Nemotron-Pretraining-SFT-v1. We incorporate *Nemotron-Pretraining-SFT-v1* [32]as a source of short-form supervised fine-tuning style pretraining data covering code, math, and general knowledge domains. For each subset, we retain only the short chain-of-thought portion of the data, discarding verbose reasoning traces. All subsets reach Darwin Level L5.

- *Nemotron-Pretraining-SFT-v1-Code.* This subset is synthesized using the Genetic-Instruct framework [40], an evolutionary algorithm that begins with a small set of seed coding instructions and iteratively generates diverse, challenging instruction-code pairs through two operations: crossover, which prompts an LLM to produce new instructions from a set of seed examples, and mutation, which evolves an existing instruction into a harder or more varied one. A separate Coder-LLM generates corresponding code solutions, and a Judge-LLM filters outputs based on correctness and quality. This process yields a large-scale collection of coding problem-solution pairs spanning multiple programming languages, reaching Darwin Level L5 and contributing approximately 21B tokens.
- *Nemotron-Pretraining-SFT-v1-Math.* This subset is synthesized following OpenMathInstruct-2 [41], which uses a strong teacher model to generate multiple candidate solutions for a diverse set of seed math questions drawn from existing benchmarks such as GSM8K and MATH. High-quality solutions are selected based on answer correctness, and question diversity is explicitly optimized to maximize coverage across mathematical topics and difficulty levels. This reaches Darwin Level L5 and contributes approximately 138B tokens.

- *Nemotron-Pretraining-SFT-v1-General.* This subset consists of MMLU-style question-answer pairs covering a broad range of knowledge topics across different domains and difficulty levels, synthesized by prompting an LLM to generate both questions and answers grounded in curated source documents. This reaches Darwin Level L5 and contributes approximately 12B tokens.

Llama-Nemotron-Post-Training-Dataset. We incorporate multiple subsets of *Llama-Nemotron-Post-Training-Dataset* [42], a large-scale post-training dataset in which prompts are sourced from public corpora or synthetically generated, and responses are synthesized by a range of open-source models. We use the code subset directly without additional processing, while for the science and math subsets we use the original prompts as input and apply rejection sampling using Qwen3-32B [9] in non-thinking mode, retaining only responses that pass correctness verification against ground-truth answers.

- *Llama-Nemotron-Post-Training-Dataset-Code.* This subset targets diverse programming tasks and problem-solving scenarios across multiple languages, with prompts sourced from public corpora or synthetically generated, and responses filtered for quality and correctness during the original dataset construction. It reaches Darwin Level L5 and contributes approximately 5B tokens.
- *Llama-Nemotron-Post-Training-Dataset-Science-RS.* This subset is obtained by applying rejection sampling to the science subset. The science subset comprises open-ended and multiple-choice questions spanning academic scientific domains including physics, biology, and chemistry. The questions are drawn from two sources: question-answer pairs extracted from StackOverflow, and synthetically generated MCQs conditioned on topic, subtopic, and difficulty level using Qwen2.5 models following the OpenMathInstruct-2 augmentation pipeline. It reaches Darwin Level L5 and contributes approximately 0.4B tokens.
- *Llama-Nemotron-Post-Training-Dataset-Math-RS.* This subset is obtained by applying rejection sampling to the science subset. The subset comprises mathematical reasoning problems spanning competition-level and general math domains, with responses synthesized by frontier open-source models. It reaches Darwin Level L5 and contributes approximately 10B tokens.

MegaMath-QA-RS. We apply rejection sampling to *MegaMath-QA* [34], the synthetic QA subset of MegaMath, in which question-answer pairs are extracted from mathematical web documents and refined to make intermediate reasoning steps explicit using an ELI5-style prompting strategy. RWe apply rejection sampling using Qwen3-32B [9] in non-thinking mode, retaining only responses whose final answers are verifiably correct against ground-truth answers, yielding approximately 9B tokens at Darwin Level L5.

MegaScience-RS. We apply rejection sampling to *MegaScience* [43], a post-training dataset covering scientific reasoning across multiple STEM disciplines, with questions drawn from textbooks and curated scientific sources. We apply rejection sampling using Qwen3-32B [9] in non-thinking mode, retaining responses that meet correctness criteria, yielding approximately 1B tokens at Darwin Level L5.

Darwin-Science-Book QA. We generate knowledge-grounded QA pairs directly from *Darwin-Science Book* using Qwen3-235B-A22B-Instruct [9]. To account for the distinct knowledge structures and expository styles across scientific disciplines, we design domain-specific prompts for each of the book domains covered in Darwin-Science, rather than applying a single universal prompt. The model is prompted to identify key knowledge points from each source passage and formulate question-answer pairs that are strictly grounded in the original text, with the constraint that every answer must be directly verifiable against its source passage. To further enhance learnability, the model is also prompted to supply intermediate reasoning steps that bridge the question to the answer, making implicit derivations explicit. The full set of domain-specific prompts is provided in The detailed prompt is provided in Appendix C.. This dataset reaches Darwin Level L5, contributing approximately 46B tokens.

Nemotron-CC-v1 Synthetic QA. We incorporate the Diverse QA subset of *Nemotron-CC-v1*, which is generated from high-quality documents selected from Common Crawl. In this pipeline, an LLM is prompted to generate question-answer pairs in multiple forms, including yes/no questions, open-ended questions, and multiple-choice questions,that probe factual information in the source text at different cognitive levels. The model is required to provide clear and concise answers while preserving concrete details such as numbers and specific facts from the original document. This grounding in high-quality web text spanning diverse general-domain topics makes the subset a broad-coverage complement to the domain-specific QA sources described above. This dataset reaches Darwin Level L5 and contributes approximately 492B tokens.

Table 4: Architectural specification of daVinci-LLM.

Parameters	∼3.09B	MLP intermediate size	11008
Layers)	36	Activation function	SwiGLU
Hidden size	2048	Positional encoding	RoPE (base $\theta = 10000$)
Attention heads	16	Max position embeddings	4096
KV heads (GQA)	2	Normalization	RMSNorm ($\epsilon = 10^{-6}$)
Head dimension	128	Tokenizer	Qwen2 tokenizer
Vocab size	151936	Precision	bfloat16

3 Training Recipe: How We Trained It

In this section, we provide a comprehensive disclosure of the pretraining process for daVinci-LLM, focusing on the technical execution and methodological transparency that define our approach. Moving beyond the industry standard of releasing only final checkpoints, we adopt a fully-open paradigm that documents the model's complete developmental trajectory from random initialization, ensuring that every design choice is traceable and reproducible. The following subsections detail our evaluation protocols, the multi-stage adaptive curriculum, and the systematic data orchestration strategies that shape the model's capabilities.

This documentation bridges the gap between opaque commercial technical reports and scale-limited academic papers, providing a transparent framework for the community to replicate our results and adapt the underlying technical workflows of pretraining.

3.1 Model Architecture

We adopt the Qwen2-based [7] transformer architecture for daVinci-LLM, training a 3B-parameter model from random initialization using a standard decoder-only causal language modeling objective. The architecture employs several modern design choices that balance computational efficiency with representational capacity: Grouped-Query Attention (GQA) [44] with 2 key-value heads shared across 16 query heads to reduce memory bandwidth while preserving attention expressiveness; SwiGLU [45] activation in the MLP layers with an expansion ratio of ∼5.4× (11008/2048) for enhanced nonlinearity; RMSNorm [46] for efficient pre-normalization; and Rotary Position Embeddings (RoPE) [47] with base frequency 10000 for length generalization. We train with a sequence length of 4096 tokens. The model utilizes a vocabulary of 151936 tokens from the Qwen2 tokenizer.

Table 4 provides the complete architectural specification. The configuration follows the Qwen2 design philosophy of prioritizing depth (36 layers) with moderate hidden dimensions (2048), a strategy that has proven effective for balancing parameter count, training throughput, and downstream task performance in the 3B scale regime.

3.2 Evaluation Protocol

Evaluation philosophy. Traditional pretraining evaluations focus solely on final performance. We adopt a more comprehensive approach aligned with our question-driven methodology: (1) Tracking capability emergence: We evaluate at 5k-step intervals to understand when different dimensions saturate. (2) Benchmark stability analysis: We systematically investigate which benchmarks remain stable indicators versus which collapse under certain training configurations (Section 4.4). (3) Multi-domain coverage: Our 19 benchmarks span general knowledge, code, and science to capture synergy effects and trade-offs.

We conduct comprehensive evaluation of our pretrained checkpoints across diverse benchmarks to assess general knowledge, reasoning, code generation, and mathematical problem solving capabilities. This section describes our evaluation protocol, benchmark selection, and baseline comparisons.

Benchmarks and metrics. We evaluate on 19 tasks spanning three capability domains: General, Code, and Science. The General cluster includes MMLU [48], MMLU-Pro [49], AGIEval [50], HellaSwag [51], TriviaQA [52], RACE [53], WinoGrande [54], OpenBookQA [55], and PIQA [56]. The Code cluster evaluates Python code synthesis through HumanEval [57], EvalPlus [58], and MBPP [59]. The Science cluster includes GSM8K [60], GSM-Plus [61], MATH [62], GPQA [63], SuperGPQA [64], MMLU-STEM, and MMLU-Pro-STEM. Detailed descriptions of all benchmarks are provided in Appendix **??**. We evaluate using the `lm-eval-harness` framework (EleutherAI) [65].

Since the evaluated models are base checkpoints, we perform inference under greedy decoding to ensure consistency across experiments. We adopt two complementary evaluation strategies based on task characteristics. *Perplexity-based* (PPL) evaluation is applied to tasks focused on multiple-choice selection

or likelihood estimation, where the model scores each candidate answer directly: PIQA (0-shot), MMLU (5-shot), OpenBookQA (5-shot), GPQA-Main (5-shot), and MMLU-STEM (5-shot). *Generative-based* evaluation is applied to tasks requiring complex reasoning chains, code generation, or Chain-of-Thought (CoT) outputs, where the model generates a free-form response: MATH (4-shot), MMLU-Pro (5-shot), SuperGPQA (5-shot), MMLU-Pro-STEM (5-shot), GSM8K (8-shot), HumanEval (0-shot), EvalPlus (0-shot), MBPP (3-shot), AGIEval (0-shot), HellaSwag (0-shot), TriviaQA (5-shot), RACE (0-shot), WinoGrande (0-shot), and GSM-Plus (5-shot).

Baselines. We compare against six open base models spanning similar parameter scales:

- OLMo-3 7B (7B) [25]: Our primary reference baseline, representative model trained by academic institutions.
- OLMo-2 7B (7B) [24]: Previous generation OLMo model.
- Qwen-3.5-4B (4B) [10]: The latest model baseline from Alibaba's Qwen family.
- Qwen-3-4B (4B) [9]: Strong baseline from Alibaba's Qwen family.
- Qwen-2.5-3B (3B) [8]: Closest parameter-matched baseline (same architecture family as our model).
- LLaMa-3.2-3B (3B) [6]: Meta's LLaMa 3.2 series at matched scale.
- Yulan-Mini-2.4B (2.4B) [66]: Another model led by academic institutions.

These baselines provide both scale-matched comparisons (3–4B parameters) and capability-matched comparisons (7B models with stronger absolute performance). Our goal is to demonstrate that principled multi-stage pretraining can enable a 3B model to approach or exceed the performance of larger baselines through careful data mixture design and training stage orchestration.

Table 5: Training hyperparameters of different training stages. Note that the Global Batch Size refers to the total number of sequences per training step; the total number of tokens per step is calculated as Global Batch Size × Sequence Length.

	Stage 1		**Stage 2**	
	Stage 1-1	Stage 1-2	Stage 2-1	Stage 2-2
Training Tokens	4 Trillion	2 Trillion	1 Trillion	1 Trillion
Global Batch Size	1024→2048→4096	4096	4096	4096
Rotatory Base	10000	10000	10000	10000
Sequence Length	4096	4096	4096	4096
Precision	bfloat16	bfloat16	bfloat16	bfloat16
Weight Decay	0.1	0.1	0.1	0.1
AdamW β_1	0.9	0.9	0.9	0.9
AdamW β_2	0.95	0.95	0.95	0.95
LR Strategy	2000 warmup + constant (3e-4)	cosine decay (3e-4→3e-5)	2000 warmup + constant (3e-5)	constant (3e-5)

3.3 Training Methodology

Building upon the architectural specifications and evaluation framework previously defined, we present the systematic pretraining methodology for daVinci-LLM. Moving beyond the view of pretraining as a monolithic sequence of token consumption, we adopt a **multi-stage, adaptive curriculum** that evolves in tandem with the model's maturing capabilities. Our training process is structured into two primary phases, shifting from expansive knowledge acquisition to reasoning-intensive enhancement:

- **Stage 1 (General Foundation Pretraining)**: Establishes broad foundational capabilities through 6T tokens of diverse, web-scale corpora. This phase is executed across two substages (Stage 1-1/1-2), utilizing progressive data adjustment to calibrate the model's exposure to web text, code, and scientific content.
- **Stage 2 (Reasoning Capability Enhancement)**: Shifts the distribution toward reasoning-dense data through an additional 2T tokens. By integrating structured QA, refined scientific content, and high-quality code, this stage amplifies reasoning capabilities, enabling our 3B-parameter model to match the 7B-scale OLMo-3.

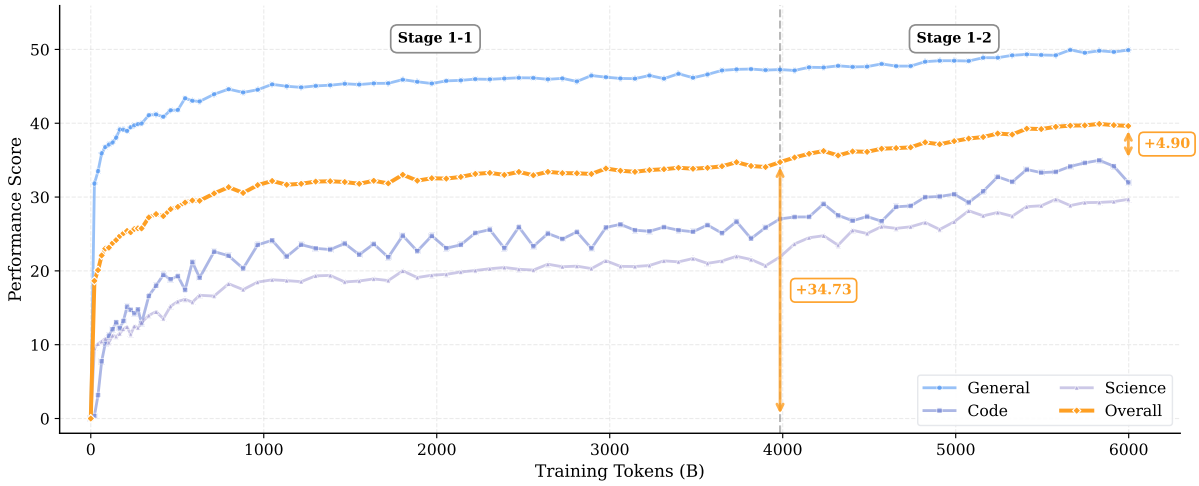


Figure 4: Progressive training results across Stage 1-1 and Stage 1-2, with checkpoints evaluated every 5000 steps. The vertical dashed line indicate the boundary between two substages.

The execution of these stages is not guided by heuristic conventions, but is principled and evidence-based, rooted in our extensive ablation studies (Section 4). Specifically, our training recipe is informed by three scientific pillars: (1) **Data Processing Depth** (Section 4.1), utilizing the Data Darwinism taxonomy to systematically enhance data quality; (2) **Adaptive Training Dynamics** (Section 4.2), where stage transitions are informed by differential saturation rates across cognitive dimensions (general knowledge plateaus early, while code and science sustain growth); and (3) **Mixture Optimization** (Section 4.3), balancing concentration for targeted enhancement with diversity for capability preservation.

3.3.1 Stage 1: General Foundation Pretraining

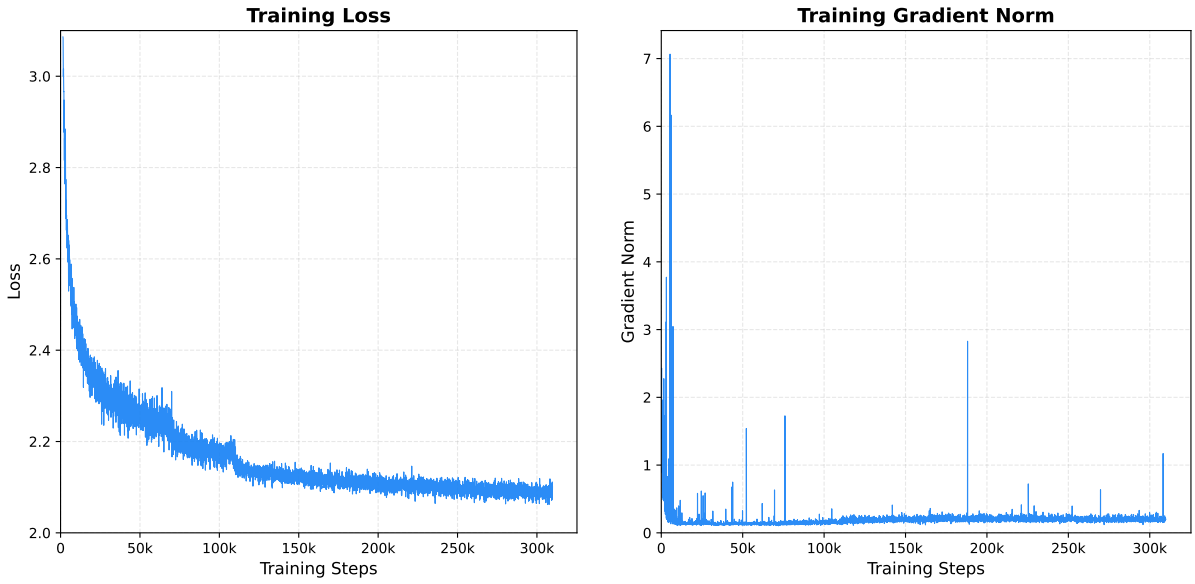
Stage 1 trains the model from random initialization across 6T tokens to establish broad foundational capabilities in natural language understanding, logical reasoning, and cross-domain knowledge synthesis.

Multi-substage Training Recipe. We decompose Stage 1 into two consecutive sub-phases: Stage 1-1 and Stage 1-2, consuming approximately 6 trillion tokens in total. This phased approach enables efficient scaling of batch size and fine-grained control over capability development through data mixture adjustments. The detailed hyperparameters for each substage are provided in Table 5 and data mixtures are provided in Table 2.

- **Stage 1-1 (Foundation Building, 4T tokens)**: This stage prioritizes stability. We employ a progressive global batch size (GBS) scaling strategy, starting at 1,024 for 70k steps, increasing to 2,048 for 40k steps, and finally reaching 4,096. The learning rate is held constant at 3e-4 after a 2,000-step linear warmup. The data mixture is dominated by Common Crawl (68.2%) to establish broad linguistic fluency.
- **Stage 1-2 (Reasoning Enhancement, 2T tokens)**: Maintains GBS at 4,096 while transitioning to cosine learning rate decay (3e-4→3e-5). To strengthen reasoning capabilities, we rebalance the mixture by reducing Common Crawl to 55.4% and increasing Code (+2.1%) and Science (+10.6%) proportions, introducing denser logical and symbolic structures.

Training Stability and Convergence. Figure 4 illustrates the training dynamics for a representative portion of Stage 1, highlighting the overall optimization stability. The training loss (Figure 5a) demonstrates smooth and consistent convergence, reflecting predictable responses to adjustments in batch size and learning rate schedules. Notably, the gradient norm (Figure 5b) remained stable throughout the illustrated steps, which is consistent with the behavior observed throughout the entire pretraining process. No significant gradient spikes or loss divergences were encountered during the run, enabling continuous training without the need for manual interventions or restarts.

Training Trajectories. Figure 4 tracks capability evolution across Stage 1's 6T-token training trajectory, with checkpoints evaluated every 5k steps across all 19 benchmarks. General knowledge benchmarks plateau rapidly within the first 1T tokens, while code and science benchmarks sustain consistent growth



(a) (b)

Figure 5: Stage 1 Training dynamics for the first 300k steps: (a) training loss curve demonstrating consistent convergence, and (b) gradient norm curve tracking optimization stability across the initial training phase.

throughout training, with notable acceleration during Stage 1-2 following the reasoning-heavy mixture adjustment. This differential saturation pattern,where general capabilities stabilize early while reasoning capabilities require extended training,motivated our multi-substage design with progressive data adjustment. Stage 1 concludes with an overall average of 39.58, establishing a solid foundation for Stage 2's reasoning enhancement.

3.3.2 Stage-2: Reasoning Capability Enhancement

Building upon the solid general language foundation established in Stage 1, Stage 2 is dedicated to transforming the model's general capabilities into high-order reasoning proficiency. Informed by the domain proportion adjustment boundary observed in Section 4.2, we recognized that merely reallocating raw text proportions was no longer sufficient to sustain continuous growth in reasoning capabilities. Consequently, Stage 2 shifts from broad linguistic modeling to structured logic acquisition by incorporating large-scale, high-density QA data and adopting a progressive curriculum learning strategy consisting of two distinct substages totaling 2T tokens.

Multi-substage Training Recipe. The training process is governed by a strategy of consolidating the foundation through balance before targeted intensification, with detailed hyperparameters and data mixtures across substages provided in Table 5 and Table 2.

- **Stage 2-1**: Balanced Foundation Building (1T tokens): To prevent overfitting or domain collapse when introduced to high-intensity reasoning data, we designed a balanced mixture consisting of structured QA, code data, and L4/L5-processed scientific data (30% each). This allocation, supplemented by 10% high-quality web text, ensures that each reasoning domain is sufficiently represented while preserving the model's general knowledge competence.
- **Stage 2-2**: QA-Intensive Enhancement (1T tokens): Building on the balanced representation established in Stage 2-1, we further increased the concentration of QA data to 70% in the final 1T tokens. This intensification strategy aims to leverage the stable representation base formed in the previous substage to amplify logical reasoning and scientific problem-solving through high-density supervisory signals.

Training Trajectories. Stage 2 trajectories demonstrate highly efficient capability gains, particularly during the transition from general foundations to specialized reasoning. As shown in Figure 6, entering the Stage 2-1 propelled the overall average from 39.58 at the Stage 1 endpoint to 48.60 within just 1T tokens. This efficiency stems from increased processing depth under the Data Darwinism framework: by utilizing L4-level Generative Refinement and L5-level Cognitive Completion, the model internalizes structured

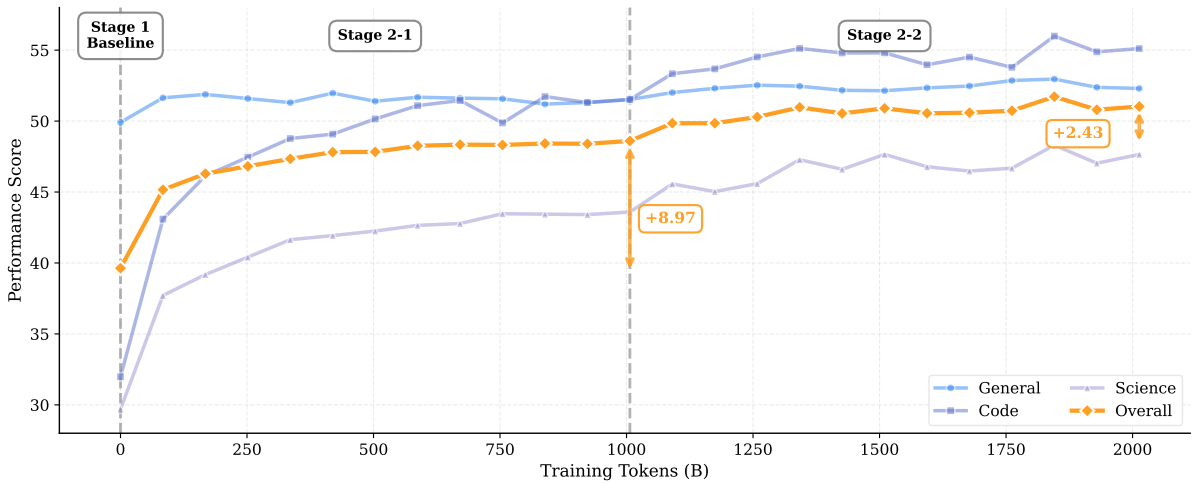


Figure 6: Progressive training results across Stage 2-1 and Stage 2-2, with checkpoints evaluated every 5000 steps. The vertical dashed line indicate the boundary between two substages.

logical chains instead of redundant narratives. Consequently, our 3B model achieved an outstanding 62.8 on the MATH benchmark, drastically exceeding its Stage 1 performance of 22.0.

These dynamics also highlight a clear stage dependency in token efficiency. Transitioning to 70% QA-intensive training in Stage 2-2 did not trigger catastrophic forgetting of general capabilities. Instead, this deliberate data mixture adjustment enabled a second acceleration in scientific reasoning, culminating in a peak overall score of 51.72. This trajectory provides empirical evidence that **in later pretraining phases, advancing data processing depth serves as a more economical scaling mechanism than simply increasing raw data volume.**

A more systematic investigation of these training dynamics, including domain-specific saturation patterns, the transition from domain adjustment to structured QA, and the resulting adaptive strategies, is presented in Section 4.2 and Section 4.3.

3.4 Final Results

Table 6 presents the comprehensive evaluation results of daVinci-3B compared to baseline models across all 19 tasks. daVinci-3B achieves 51.72 overall average, matching OLMo-3 7B despite having less than half the parameters (3B vs. 7B), and significantly outperforming parameter-matched baselines including LLaMa-3.2-3B and Yulan-Mini-2.4B.

Particularly notable is the model's strong reasoning performance: MATH exceeds the 7B-scale OLMo-3 by over 23 points; code generation achieves 55.99 average (matching OLMo-3 7B's 54.42); science reasoning reaches 48.30 (exceeding OLMo-3 7B's 45.98). Importantly, this reasoning capability enhancement is achieved while maintaining general knowledge competence comparable to larger baselines, indicating no catastrophic forgetting during specialized training.

These results validate the effectiveness of our systematic, evidence-based pretraining methodology, informed by three key findings: (1) **Data processing depth** (Section 4.1) Advancing from L3 filtering to L4 refinement to L5 synthesis enables substantial capability gains in reasoning-intensive domains, with quality enhancement outweighing naive volume scaling; (2) **Adaptive training dynamics**(Section 4.2) Monitoring differential saturation rates across capabilities and progressively adapting data composition (from domain adjustment to structured QA introduction) sustains growth beyond homogeneous data regime limits; (3) **Data Mixture optimization** (Section 4.3) Balancing aggressive reasoning-data concentration with capability preservation through adaptive composition prevents catastrophic forgetting while maximizing targeted enhancement.

Together, these findings demonstrate that systematic, question-driven investigation of pretraining dynamics can substantially improve base model capabilities. Our work shows that careful examination of data quality hierarchies, capability-specific saturation patterns, and mixture trade-offs enables more effective pretraining, suggesting significant headroom for base model improvement through principled, scientific exploration.

Table 6: Comprehensive evaluation across diverse capability benchmarks. daVinci-LLM-3B is compared against state-of-the-art open-weight models.

		Fully-open Models				*Open-weight Models*			
Domain	Benchmark	daVinci 3B	OLMO-3 7B	OLMO-2 7B	Yulan 2.4B	LLaMa-3.2 3B	Qwen-2.5 3B	Qwen-3 4B	Qwen-3.5 4B
General	MMLU	62.53	66.53	65.93	50.70	54.91	65.73	75.35	72.75
	MMLU-Pro	**43.50**	35.70	28.40	23.90	24.50	39.00	54.10	53.50
	AGIEval	26.77	33.75	31.78	28.22	22.72	37.15	45.87	44.89
	HellaSwag	71.17	74.15	80.50	68.56	73.60	73.60	73.75	75.29
	TriviaQA	49.90	55.45	68.01	27.64	55.22	51.20	47.44	49.80
	RACE	38.56	40.57	40.96	35.69	38.95	38.47	39.62	39.23
	WinoGrande	66.77	69.61	74.59	66.69	69.22	68.59	70.17	71.19
	OpenBookQA	40.20	41.80	47.60	43.00	43.00	43.80	43.20	46.60
	PIQA	77.26	78.62	81.07	76.22	77.58	78.89	77.86	78.89
	Avg General	*52.96*	*55.13*	*57.65*	*46.74*	*51.08*	*55.16*	*58.60*	*59.13*
Code	HumanEval	**61.64**	59.05	16.78	66.77	33.17	60.17	65.93	71.46
	EvalPlus	**57.32**	53.62	13.85	62.25	27.22	53.23	59.45	65.00
	MBPP	49.00	50.60	23.20	52.00	36.80	55.00	67.80	51.40
	Avg Code	***55.99***	*54.42*	*17.94*	*60.34*	*32.40*	*56.13*	*64.39*	*62.62*
Science	GSM8K	72.86	76.80	67.32	66.79	29.72	75.36	85.52	82.56
	GSM-Plus	50.38	51.58	44.58	43.71	16.12	51.21	64.17	60.04
	MATH	**62.80**	39.60	17.80	29.40	9.00	37.20	50.40	48.00
	GPQA-Main	32.37	37.05	30.80	29.91	29.46	31.47	38.17	41.07
	SuperGPQA	19.56	21.84	1.67	15.53	3.18	18.40	28.81	35.59
	MMLU-STEM	53.41	60.20	53.63	44.12	47.64	61.91	75.36	72.09
	MMLU-Pro-STEM	**46.70**	34.77	23.77	20.52	22.03	37.00	54.73	53.15
	Avg Science	***48.30***	*45.98*	*34.22*	*35.71*	*22.45*	*44.65*	*56.74*	*56.07*
Overall Average		**51.72**	51.65	42.75	44.82	37.58	51.44	58.83	58.55

4 Exploration: Why We Trained It This Way

While the preceding sections detailed the final training recipe of daVinci-LLM, this section provides the systematic investigation behind those decisions. We move beyond presenting a finalized configuration as a settled convention and instead adopt a question-driven approach to document the model's evolutionary path. By disclosing 200+ controlled ablations, we aim to elevate pretraining from an intuition-led craft to an evidence-based discipline, providing a transparent record of not only what worked, but how design choices were informed by rigorous empirical observation. This documentation serves as an empirical substrate, offering a granular view into the patterns of capability development and the strategic trade-offs encountered throughout the pretraining process.

We structure our exploration around three primary investigative themes:

1. **Data Processing Depth** (Section 4.1): We evaluate how the hierarchical progression of data processing systematically enhances model capabilities, establishing processing depth as a pivotal dimension for exploration alongside data volume scaling.
2. **Training Dynamics and Adaptation** (Section 4.2): We analyze how distinct capability saturation patterns necessitate adaptive data strategies, examining the transition from domain-proportion adjustments to structural format shifts as the effectiveness of raw text diminishes.
3. **Intensification and Preservation** (Section 4.3): We investigate the tension between targeted capability enhancement and the maintenance of general competence, identifying how compositional balance enables aggressive data intensification without triggering representational collapse.

Our investigation concludes with an analysis of **Evaluation Validity** (Section 4.4), demonstrating how evaluation protocols themselves can shape our understanding of model quality. By providing this comprehensive disclosure, we offer the empirical evidence and decision-making logic behind our recipe, contributing to a more evidence-based and systematic understanding of pretraining science.

Table 7: Comparison of rule-based and model-based filtering approaches.

Configuration	HumanEval	EvalPlus	MBPP	Avg Gen.	Avg Code	Avg Sci.	Overall
Rule-Based Filtering (L2)	54.43	48.58	42.40	51.12	48.47	40.14	46.66
Model-Based Filtering (L3)	**54.70**	48.39	**45.80**	51.19	**49.63**	**40.28**	**46.93**
Δ (L3 – L2)	+0.27	-0.19	**+3.40**	+0.07	+1.16	+0.14	+0.27

4.1 Data Processing Depth: From Filtering to Synthesis

Section Takeaway

Hierarchical data processing, progressing from quality filtering to content refinement to cognitive synthesis, enables systematic capability development. Within the Darwin framework, L3 model-based filtering yields modest but consistent gains, validating progression beyond rule-based approaches. L4 generative refinement delivers substantial improvements for complex reasoning, demonstrating that content transformation outweighs volume expansion. L5 cognitive completion enables targeted capability steering through domain-aligned synthesis. **These results establish processing depth as a systematic optimization dimension, offering a principled alternative to naive scaling**: hierarchical data processing can substitute for multi-fold data volume increases.

While data quality's importance for reasoning capabilities is increasingly recognized, the field lacks systematic understanding of *how processing depth shapes effectiveness*. Section 2.1 introduced the Data Darwinism L0-L9 taxonomy, characterizing operations from basic filtering to active synthesis. Yet a critical question remains unanswered: ***Does data processing depth systematically improve reasoning capabilities, and what effectiveness patterns vary across the hierarchy?*** Answering this determines when practitioners should invest in advanced processing versus expanding data volume, and whether systematic quality enhancement can substitute for massive scale increases.

We conduct controlled ablations across three hierarchical levels, holding data volume and training compute constant to isolate processing depth's causal effect. L3 model-based filtering (Section 4.1.1) removes low-quality content via LLM-based classifiers, preserving existing material unchanged. L4 generative refinement (Section 4.1.2) employs frontier LLMs to transform content—extracting key concepts, removing noise, reorganizing structure—while maintaining semantic integrity. L5 cognitive completion (Section 4.1.3) actively synthesizes new reasoning chains through domain-specific QA generation, moving beyond refinement to targeted capability steering. Our experiments reveal distinct effectiveness patterns: L3 provides consistent but modest improvements across tasks, with notable gains on foundational programming (+3.40 on MBPP); L4 delivers substantial gains on complex reasoning (+7.00 on MATH), though effects remain task-specific; L5 enables domain-targeted steering, with synthetic QA exhibiting strong source-target alignment but limited cross-domain transfer. These findings establish a systematic pathway from filtering to synthesis for data-centric capability improvement.

4.1.1 Code Data Filtering: L3 Model-Based Filtering

To evaluate whether model-based filtering (L3) provides measurable improvements over rule-based approaches (L2), we compare their effectiveness on code generation tasks. We apply GPT-OSS-120B quality scoring following SeedCoder's assessment framework, which identifies and removes low-quality code artifacts: configuration files with extensive hard-coded data, data files dominated by constants, code with minimal logic, and auto-generated content. We train for 500B tokens, comparing L3 model-based filtering against the L2 rule-based baseline.

Table 7 shows that L3 model-based filtering provides consistent but modest improvements across most metrics. The most notable gain appears on MBPP, while improvements on HumanEval and EvalPlus remain minimal. This differential pattern, where foundational programming tasks benefit more substantially than advanced algorithmic challenges, suggests that quality filtering's impact depends on task complexity. One plausible interpretation is that basic programming competence is more sensitive to training example clarity, as models acquiring fundamental patterns benefit from removing noisy artifacts like configuration boilerplate. However, the modest overall magnitude indicates that advancing from L2 to L3 yields incremental rather than transformative gains.

These results validate that model-based quality assessment (L3) provides measurable advantages over pure rule-based approaches (L2), confirming the value of advancing processing depth within the Darwin framework. However, the limited improvements, particularly on sophisticated coding benchmarks, suggest

Table 8: Comparison of baseline and generative refinement approaches.

Configuration	GSM-8K	GSM-Plus	MATH	Avg Gen.	Avg Code	Avg Sci.	Overall
Baseline	64.06	40.58	38.00	**51.69**	49.80	40.52	47.27
Generative Refined (L4)	**65.43**	**42.38**	**45.00**	51.40	**50.15**	**42.25**	**47.83**
Δ (L4 – Baseline)	+1.37	+1.80	**+7.00**	-0.29	+0.35	+1.73	+0.56

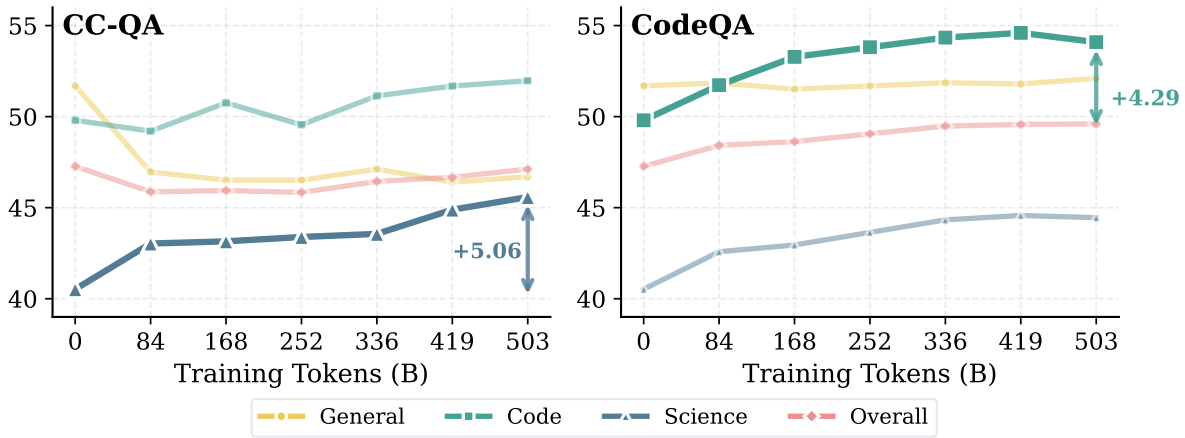


Figure 7: Effectiveness of CC-QA and CodeQA synthetic data in Stage 2-2.

that filtering alone may be insufficient to unlock the full potential of code training data. More intensive processing interventions, such as generative refinement to improve code clarity and documentation, may be necessary to achieve breakthrough gains in code capabilities.

4.1.2 Math Data Quality: L4 Generative Refinement

Section 4.1.1 indicates that L3 filtering offers marginal returns, prompting an investigation into whether active content transformation (L4) can further enhance reasoning performance. To test this, we compared L4-refined mathematical data against the original MegaMath and Nemotron-CC-Math baseline, holding data volume constant at 500B tokens. We utilize Qwen3-235B to perform generative refinement, which systematically prunes narrative noise, extracts core formulas, and reorganizes fragmented steps into logically structured proofs while strictly preserving semantic integrity.

As shown in Table 8, L4 refinement yields substantial improvements across all mathematical benchmarks, though the gains are markedly uneven. The MATH benchmark achieved a significant +7.00 increase, while the improvement on GSM8K was relatively modest (+1.37). This divergence suggests that structural purification, specifically the transition from messy exposition to pedagogical logic, disproportionately benefits complex, multi-step reasoning. While basic word problems in GSM8K follow relatively simple and linear solution paths, the abstract problems in MATH are highly sensitive to the clarity of the underlying logical flow. By stripping away redundant exposition and reinforcing the connection between reasoning steps, L4 processing effectively reduces the difficulty for models to extract fundamental patterns from complex mathematical text, leading to a performance leap in advanced reasoning tasks.

4.1.3 L5 Synthetic QA: Cognitive Completion

While L4 refinement reorganizes existing content, L5 cognitive completion involves the active synthesis of new reasoning chains to bridge the gap between implicit information and explicit problem-solving steps. We investigate L5 processing by incorporating synthetic QA data from two domain-specific sources: CodeQA, derived from curated repositories, and CC-QA, extracted from high-quality web text. Unlike previous levels that modify existing material, L5 synthesis allows generative models to construct structured reasoning scaffolds that may be absent in raw corpora.

The experimental results in Figure 7 reveal a strong pattern of domain-specific steering: CodeQA substantially enhances programming benchmarks, while CC-QA primarily strengthens science reasoning. This source-target alignment suggests that the impact of synthetic reasoning is largely governed by its originating domain. For instance, code-domain QA synthesizes algorithmic structures that directly benefit programming but offer limited cross-domain transfer to unrelated areas like general knowledge or science.

Figure 8: Stage 1 benchmark performance. Light curves show individual benchmarks; dark curves show category averages for general knowledge, code, and science. Two vertical dashed lines indicate the boundaries between substages 1-1, 1-2, and 1-3, respectively.

This limited generalization indicates that L5 synthesis acts as a high-precision tool for capability steering rather than a universal performance booster.

These findings establish that strategic synthesis can be as effective as massive data scaling. By generating data that mirrors specific reasoning structures, practitioners can deliberately guide model performance toward target domains. This positions L5 processing as a principled alternative to exhaustive data accumulation, offering a more efficient path to capability development.

4.2 Training Dynamics: Adaptive Data Strategies

Section Takeaway

Effective pretraining requires stage-specific data strategies guided by capability dynamics. Systematic convergence tracking reveals that different capabilities exhibit vastly different saturation timescales, enabling adaptive interventions that reallocate compute toward actively learning capabilities. However, domain proportion adjustments encounter fundamental limitations once standard corpus formats collectively approach saturation, at which point, reallocating among these data types no longer suffices. Stage 1's diminishing returns from proportion adjustments motivated introducing structured question-answer data in Stage 2, which substantially outperforms continued proportion optimization. These findings establish that **no single data mixture or format suffices across extended training**: sustained capability development demands monitoring convergence patterns and adapting both domain proportions and data formats accordingly.

Section 4.1 established that advancing data processing depth enables systematic quality improvements. However, even with high-quality processed data, how to compose them optimally remains a critical question. Traditional approaches apply uniform strategies with fixed data mixtures over predetermined token budgets, implicitly assuming all capability dimensions develop at similar timescales. Yet if general knowledge saturates rapidly while reasoning capabilities require sustained training, maintaining static mixtures may waste compute on converged dimensions while under-serving actively-learning ones.

This motivates our central question: ***How should data composition evolve as different capabilities mature at different rates?*** Specifically, we investigate two aspects: (1) Do different capability dimensions exhibit systematically different saturation patterns? We investigate whether tracking domain-specific convergence enables principled intervention timing and adaptive data composition adjustments. (2) What adaptation strategies work when capabilities diverge? We examine whether domain proportion adjustment suffices, or whether fundamental data format shifts become necessary as standard corpus formats collectively approach saturation.

We address these through systematic convergence tracking across Stage 1 (6T tokens) and Stage 2 (2T tokens). Section 4.2.1 examines domain-specific saturation dynamics within Stage 1, revealing differential convergence rates (general knowledge plateaus at 1T tokens; code/science sustain growth through 4T tokens) that motivate adaptive domain proportion adjustments and the boundaries of such strategies. Section 4.2.2 investigates the transition from domain adjustment to structured QA introduction in Stage 2, establishing when proportion optimization suffices versus when data format shifts become necessary.

4.2.1 Domain Proportion Adjustment

Do different capability dimensions saturate at different rates, and can we use this to guide adaptive training? We investigate this through Stage 1's training trajectory (6T tokens), tracking capability-specific

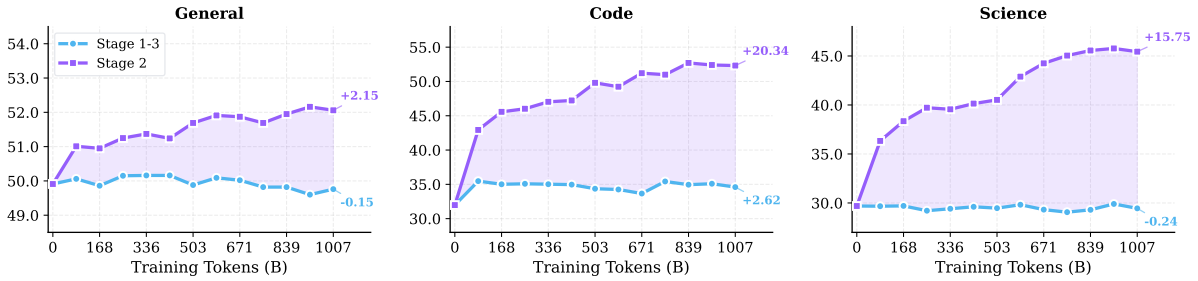


Figure 9: Performance comparison between continued data domain adjustment (stage 1-3) and QA introduction (Stage 2).

convergence patterns across general knowledge, code, and scientific domains. We examine whether domain proportion adjustments, reallocating compute from saturated to actively-learning capabilities, can sustain capability growth, and where such strategies encounter fundamental limitations.

Differential convergence motivates adaptive mixture adjustment. Figure 8 shows Stage 1's 6T-token trajectory, revealing distinct convergence patterns across data domains. General knowledge benchmarks plateau rapidly within the first 1T tokens, with minimal subsequent improvement. In contrast, reasoning-intensive benchmarks, both code and scientific domains, sustain consistent growth through the initial 4T tokens, though improvement rates begin to slow. Notably, benchmarks within each domain exhibit consistent convergence behavior: all general knowledge tasks saturate early, while all reasoning tasks maintain growth trajectories, confirming these patterns reflect domain-level characteristics rather than task-specific idiosyncrasies.

This differential saturation motivated Stage 1-2's adaptive adjustment strategy at the 4T checkpoint. With general knowledge approaching saturation while reasoning capabilities remained actively improving, we reduced generic web data proportion and increased code/science concentrations, complemented by transitioning to cosine learning rate decay. Figure 8 shows that this adjustment successfully amplified reasoning domain gains in Stage 1-2 (4-6T tokens): both code and science benchmarks exhibit renewed acceleration, while general knowledge maintains stable performance without degradation. These results validate adaptive mixture strategies that reallocate compute from saturated domains (general knowledge) toward capabilities exhibiting sustained learning potential (code and science reasoning).

Domain adjustment encounters saturation boundaries. Encouraged by Stage 1-2's success, we explored further domain proportion adjustments, referred to as Stage 1-3, in Stage 1's later phase to sustain reasoning capability growth. As code and science performance began to decelerate again near 6T tokens, we attempted additional increases in code/science concentrations while further reducing generic web data.[1] However, figure 8 shows that these adjustments yielded only marginal improvements at Stage 1-3, with performance gains substantially smaller than those achieved during Stage 1-2's adjustment. This diminishing effectiveness reveals an inherent limitation: once standard pretraining corpora (web, code, science) collectively approaches saturation, reallocating proportions among these textual formats cannot overcome the fundamental constraint. These observations suggest that sustaining growth requires introducing new data formats, a transition we investigate in the following section.

4.2.2 From Domain Adjustment to QA Introduction

Section 4.2.1 demonstrated that domain proportion adjustments encounter diminishing returns as standard pretraining corpora approach saturation. To overcome this limitation, we hypothesize that sustaining capability growth requires shifting from raw text to data with explicit reasoning structures. We therefore introduce structured question-answer pairs in Stage 2. Unlike raw text, QA data provides inherent problem-solving scaffolds: questions define the reasoning target, while answers supply direct supervision signals for multi-step inference. This format is uniquely suited for reasoning-intensive domains, precisely the domains that Section 4.2.1 showed require extended training investment.

Structured QA outperforms domain adjustment alone. To isolate the value of introducing QA data, we conducted a controlled comparison. Starting from the Stage 1-2 checkpoint, we examined two continuation paths: (1) Stage 1-3, which further increases code/science proportions without adding

[1] Stage 1-3 configuration: CC 51.42%, Code 13.64%, Math 13.58%, Science 21.36%, with constant learning rate 3e-5.

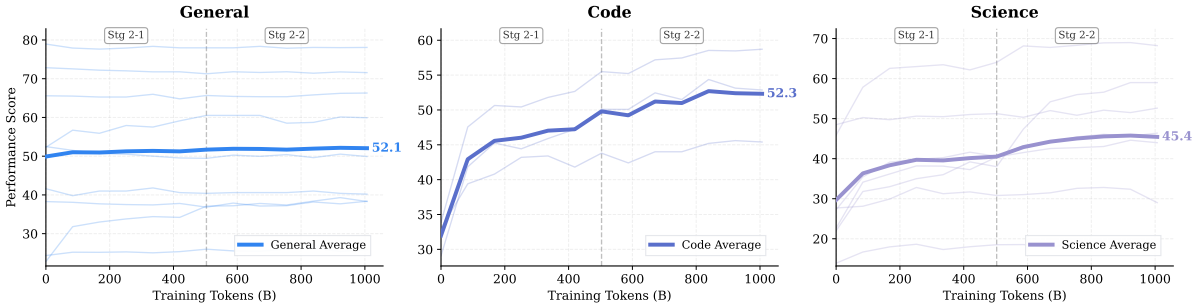


Figure 10: Stage 2 benchmark performance. Light curves show individual benchmarks; dark curves show category averages for general knowledge, code, and science. The vertical dashed lines indicate the boundaries between Substages 2-1, 2-2.

QA (as described in Section 4.2.1), versus (2) Stage 2, which introduces 30% QA. Figure 9 presents the performance trajectories. Stage 2 substantially outperforms Stage 1-3 across reasoning domains: code and scientific benchmarks achieve significantly higher performance with QA introduction compared to mere domain adjustment. While general knowledge also exhibits a modest improvement, the gains are disproportionately concentrated in reasoning-intensive capabilities. This validates that structured data formats, rather than simple proportion adjustments, are essential to overcome the saturation of unstructured text regimes and unlock advanced cognitive potential.

Adaptive Adjustment within Stage 2 sustains growth. Having established QA's effectiveness, we examine Stage 2's internal dynamics. Figure 10 reveals convergence patterns consistent with Stage 1's hierarchy: general knowledge benchmarks exhibit near-complete saturation throughout Stage 2, showing minimal variation. In contrast, the code and scientific reasoning domains achieve substantial performance gains during Stage 2. When improvement rates for these domains begin to decelerate toward the end of Stage 2-1, we implement further mixture adjustment in Stage 2-2, increasing QA concentration to 70%. This successfully triggers renewed performance growth, demonstrating that adaptive mixture adjustment remains effective across training stages, with detailed mixture design explored in Section 4.3.

Implementation details beyond mixture design. In addition to mixture adjustment, we examined two practical training decisions that could plausibly affect Stage 2 outcomes: the learning rate schedule in later substages and the masking policy for QA data. Empirically, cosine decay provides small but consistent gains over a constant learning rate, whereas masking question tokens yields only marginal improvements when the QA data is already diverse and high quality. We defer the full analysis to Appendix B.

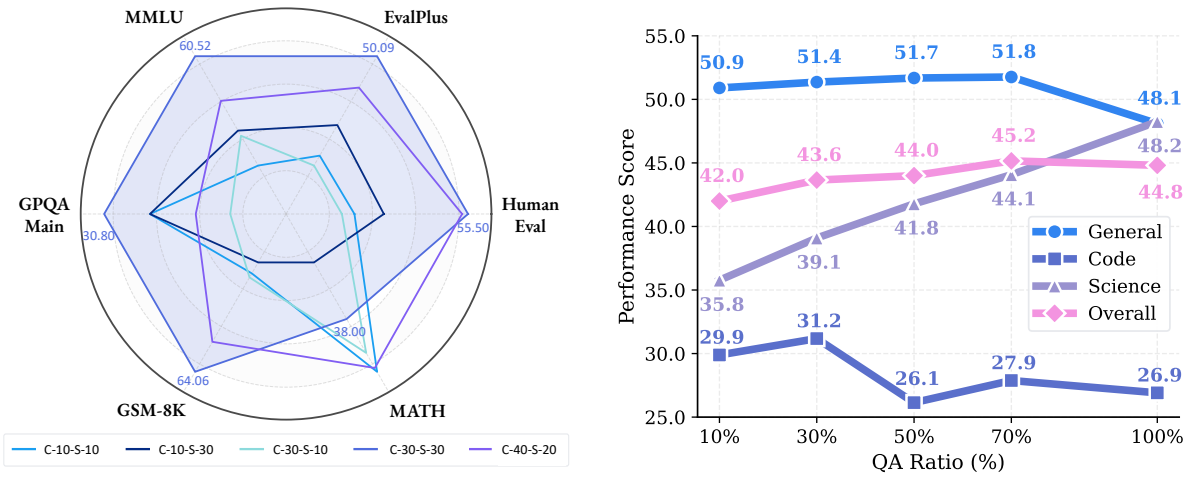
4.3 Data Mixture Design: Balancing and Intensifying

Section Takeaway

Mixture optimization balances reasoning intensification with global capability preservation. Our results show that while high reasoning data concentrations are essential, internal balance between domains is critical to prevent over-specialization sacrificing broader competence. Similarly, structured QA intensification requires navigating a stage-dependent tolerance: conservative ratios preserve stability during foundational training, while progressive concentration can be leveraged once a balanced representational base is established. These findings demonstrate that **concentration and preservation are not mutually exclusive, as effective mixture design achieves both through an adaptive composition that evolves with training progress.**

Section 4.2 established that transitioning to structured reasoning formats is essential for surpassing the performance plateaus of unstructured text. However, the introduction of these data types raises a new challenge: the tension between **targeted intensification** and **capability preservation**. While aggressive concentration of reasoning-intensive data is necessary to drive performance, it also risks triggering catastrophic trade-offs where gains in one domain come at the expense of others. This leads to a critical research question: ***How to manage the intensification of reasoning data to maximize gains without compromising the model's overall capability breadth?***

We address this question through two ablation studies. Section 4.3.1 examines the internal composition of reasoning domains, documenting how an aggressive yet balanced allocation between code and science



(a) (b)

Figure 11: Code and science data ratio analysis. Notation: C-X-S-Y indicates X% code and Y% science. (a) Performance across code and science proportions (QA fixed at 30%, remainder allocated to CommonCrawl web data). (b) Data synergy effects across QA ratios in Stage 2-1.

prevents over-specialization and yields more robust results than extreme domain-specific concentrations. Section 4.3.2 investigates the progressive intensification of structured QA data. We demonstrate that the model's tolerance for high-concentration supervision is stage-dependent: a conservative mixture is required to maintain stability during foundational training (Stage 2-1), whereas an intensified regime can be successfully applied (Stage 2-2) once a balanced capability base has been established.

4.3.1 Domain Balance: Code and Science Composition

Section 4.2.1 reveals that different domains exhibit distinct convergence rates during pretraining. Building on this observation, we investigate the internal composition of reasoning-intensive data (specifically code and science) when training jointly with QA. We fix QA concentration at 30% and systematically vary code-science proportions: 10%-10%, 10%-30%, 30%-10%, 30%-30%, and 40%-20%, with the remainder allocated to CommonCrawl web data. All configurations are trained for 500B tokens from the Stage 1 checkpoint. We evaluate performance across general knowledge, code generation, and science reasoning, visualizing the results via radar plots (Figure 11a).

Two critical patterns emerge from this ablation. First, **reasoning-intensive intensification is essential for Stage 2 progress**: configurations allocating 60% to code and science (C-30-S-30 and C-40-S-20) substantially outperform lower concentrations. This validates that transitioning from Stage 1's CC-dominated mixture to Stage 2's reasoning-heavy regime requires aggressive reallocation toward symbolic and logical domains. Second, **within high-concentration regimes, internal balance is critical for maintaining overall capability**: C-30-S-30 achieves superior overall performance compared to the code-heavy C-40-S-20. This indicates that while intensification is necessary, extreme specialization in a single domain can trigger trade-offs, whereas equal allocation between code and science maintains general competence while delivering competitive reasoning gains across all dimensions.

These findings establish the design principle for Stage 2-1: aggressive reasoning-data concentration with internal balance prevents under-resourcing of specific reasoning domains while preserving general capabilities. The balanced configuration (30% QA, 30% code, 30% science, 10% web text) forms the foundation for subsequent QA intensification strategies examined in the next section.

4.3.2 QA Concentration: Progressive Intensification

Section 4.3.1 established balanced proportions within the reasoning domains. We now investigate the concentration of structured QA itself, revealing stage-dependent optimal ratios: conservative choices in Stage 2-1 prioritize stability, while higher concentrations in Stage 2-2 leverage established foundations to safely amplify reasoning gains.

Stage 2-1: Conservative QA choice prioritizes capability balance. We conduct a systematic investigation in Stage 2-1, training configurations with 10% to 100% QA. As shown in Figure 11b, general knowledge remains stable across moderate concentrations but degrades sharply at 100% QA, indicating

Table 9: Impact of QA data ratio on model performance in Stage 2-2.

Configuration	Training Tokens	General	Code	Science	Overall
QA-30%	84B	51.59	47.71	40.25	46.80
	252B	51.66	46.79	40.03	46.61
	419B	51.80	47.16	40.59	46.94
QA-50%	84B	51.72	49.86	41.60	47.70
	252B	51.85	49.01	42.80	48.06
	419B	51.83	49.89	43.42	48.43
QA-70%	84B	51.91	49.23	42.89	48.17
	252B	51.69	50.99	45.04	49.13
	419B	**52.16**	**52.40**	**45.77**	**49.84**

that exclusive structured exposure cannot sustain broad linguistic competence. Notably, code performance exhibits a non-monotonic behavior: improving initially but collapsing beyond the 30% threshold. In contrast, science reasoning improves monotonically.

This divergence between science and code merits further investigation. We hypothesize that this stems from the compositional imbalance of our QA corpus: our Stage 2-1 QA pool is heavily weighted toward science (approx. 80%) , while code-related QA is relatively scarce (approx. 26B tokens). At higher total QA concentrations, the insufficient diversity of code samples may trigger premature over-fitting or representation collapse, whereas the abundant science data sustain growth. This suggests a critical requirement for scaling QA: performance is governed not just by the total ratio, but by the absolute diversity of the supervised signals in each domain.

In our final Stage 2-1 recipe, we select 30% QA to maintain balanced capabilities. Although 70% QA yields higher average peak scores, the resulting code degradation suggests that a conservative foundation-building phase is essential to prevent irreversible domain loss before further intensification.

Stage 2-2: Foundation enables aggressive QA intensification without collapse. Continuing from the Stage 2-1 checkpoint, we investigated higher QA concentrations in Stage 2-2 (30%, 50%, and 70%). In this phase, performance increases monotonically across all clusters, including code. This contrasts sharply with Stage 2-1, where increasing QA beyond 30% triggered collapse. We hypothesize that Stage 2-1's balanced training establishes a necessary foundation that enables the model to effectively internalize high-intensity supervision without capability loss.

These findings validate a progressive mixture strategy: Stage 2-1 employs a conservative 30% QA ratio to build stability, while Stage 2-2 intensifies to 70% QA for targeted enhancement. This staged approach achieves what static mixtures cannot by preserving broad competence while delivering significant reasoning gains. The complete trajectory demonstrates that no single mixture suffices: sustained development requires adapting data composition across training stages.

4.4 Evaluation Validity: PPL-based vs. Generative-based

Takeaway

Evaluation design critically impacts conclusions drawn about base model pretraining: The choice between PPL-based and generative evaluation is not merely technical—these protocols probe different aspects of model capability, and models with extensive QA pretraining can exhibit ranking reversals across them. Benchmark selection and evaluation methodology must therefore be validated jointly against the intended use case.

Previous sections focused on the *training side*: how data mixture design, QA proportion, and data quality shape model behavior during continued pretraining. We now turn to the *evaluation side*. Even when training settings are fixed, the conclusions we draw about model quality can vary substantially depending on how the model is evaluated. In particular, base models are commonly assessed under two distinct protocols: **PPL-based evaluation**, which measures whether the model assigns higher likelihood to the correct answer, and **generative evaluation**, which requires the model to actively produce an answer. These protocols are often treated as interchangeable, but they in fact probe different aspects of capability and can lead to different model rankings. Therefore, it is necessary to clarify what each evaluation setting is actually measuring and which one better matches the intended use case.

Figure 12 illustrates this difference on MMLU across three representative base models. Under PPL-based evaluation, OLMO-2-7B slightly outperforms Qwen-2.5-3B; under generative evaluation, the ranking

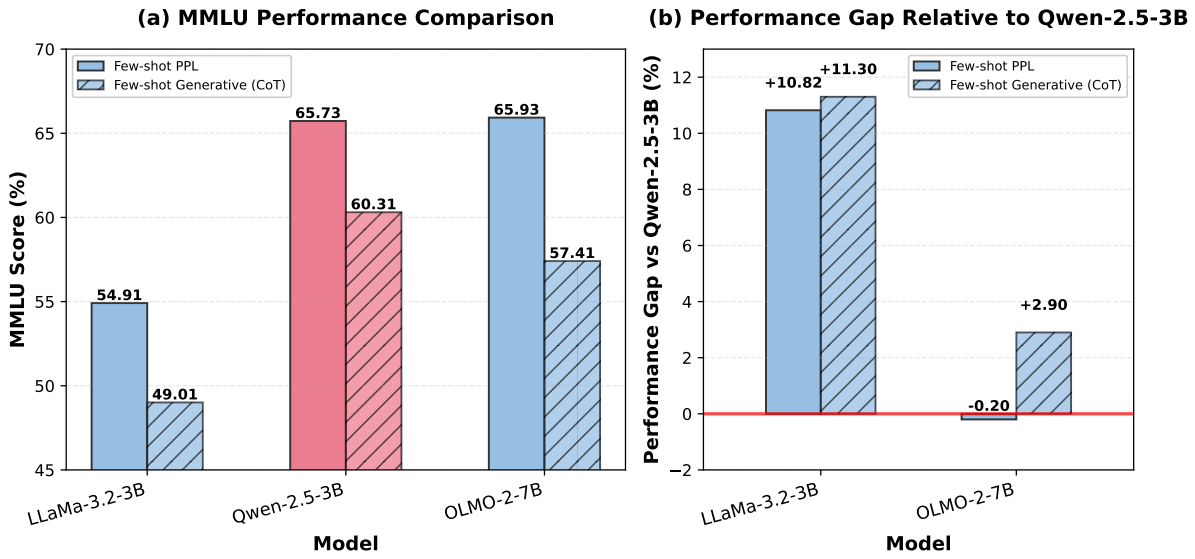


Figure 12: MMLU performance comparison across evaluation protocols. Panel (a) shows absolute scores under PPL-based and generative (CoT) evaluation. Panel (b) highlights the performance gap relative to Qwen-2.5-3B, demonstrating how models with extensive QA training exhibit amplified advantages in generative settings. The gap between OLMO-2-7B and Qwen-2.5-3B increases from -0.20% (PPL) to +2.90% (generative), a 3.10% swing that reverses the ranking.

reverses in Qwen-2.5-3B's favor—a 3.10% swing. We argue that this discrepancy arises because the two protocols place different demands on the model. **PPL-based evaluation is closer to latent knowledge access**: the model only needs to assign relatively higher probability to the correct candidate among presented options. By contrast, **generative evaluation requires the model to surface and organize knowledge into an explicit answer**, often under additional reasoning demands. In this sense, generative evaluation is more sensitive not only to what the model knows, but also to whether it has learned to operationalize that knowledge through answer production. This distinction helps explain why models with heavier QA exposure during pretraining tend to gain disproportionately under generative evaluation. QA-style pretraining does not merely inject factual content; it also trains the model in the behavioral format of mapping questions to explicit answers. As a result, two models with similar underlying knowledge may appear much closer under PPL-based evaluation, yet diverge substantially when evaluated generatively. The ranking reversal between OLMO-2-7B and Qwen-2.5-3B is a concrete example of this effect.

Framing one protocol as inherently superior would therefore be misleading. The appropriate choice depends on the deployment scenario. Applications requiring direct answer generation–such as chatbots or open-ended QA systems–are better matched by generative evaluation, whereas applications that use language models as scoring or ranking functions are better aligned with PPL-based evaluation. More broadly, when comparing base models, substantial discrepancies across protocols should not be dismissed as noise: they often indicate meaningful differences in pretraining data composition, especially QA exposure, and should be reported transparently. Evaluating base models under both protocols provides a more complete capability profile and helps avoid protocol-induced ranking artifacts.

5 Conclusion

We have presented daVinci-LLM, a systematic investigation of pretraining dynamics through controlled experimentation and comprehensive transparency. Through the Data Darwinism framework and 200+ controlled ablations, we establish empirical understanding of data processing depth, training dynamics, and mixture design, releasing complete pipelines and exploration results to the community. This work represents a step toward transparency, and transforming pretraining into rigorous scientific discipline requires community-wide open collaboration. Only when systematic exploration becomes the norm and empirical knowledge accumulates across organizations can pretraining advance from intuition-driven practice toward genuine scientific understanding.

References

[1] OpenAI. Introducing gpt-5.2. https://openai.com/zh-Hans-CN/index/introducing-gpt-5-2/, December 2025. Accessed: 2026-02-07.

[2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[3] Anthropic. Introducing claude opus 4.6. https://www.anthropic.com/news/claude-opus-4-6, February 2026. Accessed: 2026-02-07.

[4] Google. A new era of intelligence with gemini 3. https://blog.google/products-and-platforms/products/gemini/gemini-3/, November 2025. Accessed: 2026-02-07.

[5] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.

[6] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407, 2024.

[7] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

[8] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[9] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[10] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL https://qwen.ai/blog?id=qwen3.5.

[11] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[12] Nathan Lambert. The atom project, August 2025. URL https://atomproject.ai.

[13] Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*, 2023.

[14] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[15] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

[16] Yiwei Qin, Xuefeng Li, Haoyang Zou, Yixiu Liu, Shijie Xia, Zhen Huang, Yixin Ye, Weizhe Yuan, Hector Liu, Yuanzhi Li, et al. O1 replication journey: A strategic progress report–part 1. *arXiv preprint arXiv:2410.18982*, 2024.

[17] Zhen Huang, Haoyang Zou, Xuefeng Li, Yixiu Liu, Yuxiang Zheng, Ethan Chern, Shijie Xia, Yiwei Qin, Weizhe Yuan, and Pengfei Liu. O1 replication journey–part 2: Surpassing o1-preview through simple distillation, big progress or bitter lesson? *arXiv preprint arXiv:2411.16489*, 2024.

[18] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*, 2025.

[19] Shijie Xia, Yiwei Qin, Xuefeng Li, Yan Ma, Run-Ze Fan, Steffi Chern, Haoyang Zou, Fan Zhou, Xiangkun Hu, Jiahe Jin, et al. Generative ai act ii: Test time scaling drives cognition engineering. *arXiv preprint arXiv:2504.13828*, 2025.

[20] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

[21] Yixiu Liu, Yang Nan, Weixian Xu, Xiangkun Hu, Lyumanshan Ye, Zhen Qin, and Pengfei Liu. Alphago moment for model architecture discovery. *arXiv preprint arXiv:2507.18074*, 2025.

[22] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys*, 55(9):1–35, 2023.

[23] Qishuo Hua, Lyumanshan Ye, Dayuan Fu, Yang Xiao, Xiaojie Cai, Yunze Wu, Jifan Lin, Junfei Wang, and Pengfei Liu. Context engineering 2.0: The context of context engineering. *arXiv preprint arXiv:2510.26493*, 2025.

[24] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 olmo 2 furious. *arXiv preprint arXiv:2501.00656*, 2024.

[25] Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, et al. Olmo 3. *arXiv preprint arXiv:2512.13961*, 2025.

[26] Syeda Nahida Akter, Shrimai Prabhumoye, Eric Nyberg, Mostofa Patwary, Mohammad Shoeybi, Yejin Choi, and Bryan Catanzaro. Front-loading reasoning: The synergy between pretraining and post-training data. *arXiv preprint arXiv:2510.03264*, 2025.

[27] Oded Ovadia, Menachem Brief, Moshik Mishaeli, and Oren Elisha. Fine-tuning or retrieval? comparing knowledge injection in llms. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 237–250, 2024.

[28] Sergey Pletenev, Maria Marina, Daniil Moskovskiy, Vasily Konovalov, Pavel Braslavski, Alexander Panchenko, and Mikhail Salnikov. How much knowledge can you pack into a lora adapter without harming llm? *arXiv preprint arXiv:2502.14502*, 2025.

[29] Yiwei Qin, Zhen Huang, Tiantian Mi, Weiye Si, Chenyang Zhou, Qipeng Guo, Siyuan Feng, and Pengfei Liu. Data darwinism part i: Unlocking the value of scientific data for pre-training. *arXiv preprint arXiv:2602.07824*, 2026.

[30] Dan Su, Kezhi Kong, Ying Lin, Joseph Jennings, Brandon Norick, Markus Kliegl, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-cc: Transforming common crawl into a refined long-horizon pretraining dataset. *arXiv preprint arXiv:2412.02595*, 2024.

[31] Siming Huang, Tianhao Cheng, J. K. Liu, Jiaran Hao, Liuyihan Song, Yang Xu, J. Yang, Jiaheng Liu, Chenchen Zhang, Linzheng Chai, Ruifeng Yuan, Zhaoxiang Zhang, Jie Fu, Qian Liu, Ge Zhang, Zili Wang, Yuan Qi, Yinghui Xu, and Wei Chu. Opencoder: The open cookbook for top-tier code large language models, 2025. URL https://arxiv.org/abs/2411.04905.

[32] NVIDIA, :, Aarti Basant, Abhijit Khairnar, Abhijit Paithankar, Abhinav Khattar, Adithya Renduchintala, Aditya Malte, Akhiad Bercovich, Akshay Hazare, Alejandra Rico, Aleksander Ficek, Alex Kondratenko, Alex Shaposhnikov, Alexander Bukharin, Ali Taghibakhshi, Amelia Barton, Ameya Sunil Mahabaleshwarkar, Amy Shen, Andrew Tao, Ann Guan, Anna Shors, Anubhav Mandarwal, Arham Mehta, Arun Venkatesan, Ashton Sharabiani, Ashwath Aithal, Ashwin Poojary, Ayush Dattagupta, Balaram Buddharaju, Banghua Zhu, Barnaby Simkin, Bilal Kartal, Bita Darvish Rouhani, Bobby Chen, Boris Ginsburg, Brandon Norick, Brian Yu, Bryan Catanzaro, Charles Wang, Charlie Truong, Chetan Mungekar, Chintan Patel, Chris Alexiuk, Christian Munley, Christopher Parisien, Dan Su, Daniel Afrimi, Daniel Korzekwa, Daniel Rohrer, Daria Gitman, David Mosallanezhad, Deepak Narayanan, Dima Rekesh, Dina Yared, Dmytro Pykhtar, Dong Ahn, Duncan Riach, Eileen Long, Elliott Ning, Eric Chung, Erick Galinkin, Evelina Bakhturina, Gargi Prasad, Gerald Shen, Haifeng Qian, Haim Elisha, Harsh Sharma, Hayley Ross, Helen Ngo, Herman Sahota, Hexin Wang, Hoo Chang Shin, Hua Huang, Iain Cunningham, Igor Gitman, Ivan Moshkov, Jaehun Jung, Jan Kautz, Jane Polak Scowcroft, Jared Casper, Jian Zhang, Jiaqi Zeng, Jimmy Zhang, Jinze Xue, Jocelyn Huang, Joey Conway, John Kamalu, Jonathan Cohen, Joseph Jennings, Julien Veron Vialard, Junkeun Yi, Jupinder Parmar, Kari Briski, Katherine Cheung, Katherine Luna, Keith Wyss, Keshav Santhanam, Kezhi Kong, Krzysztof Pawelec, Kumar Anik, Kunlun Li, Kushan Ahmadian, Lawrence McAfee, Laya Sleiman, Leon Derczynski, Luis Vega, Maer Rodrigues de Melo, Makesh Narsimhan Sreedhar, Marcin Chochowski, Mark Cai, Markus Kliegl, Marta Stepniewska-Dziubinska, Matvei Novikov, Mehrzad Samadi, Meredith Price, Meriem Boubdir, Michael Boone, Michael Evans, Michal Bien, Michal Zawalski, Miguel Martinez, Mike Chrzanowski, Mohammad Shoeybi, Mostofa Patwary, Namit Dhameja, Nave Assaf, Negar Habibi, Nidhi Bhatia, Nikki Pope, Nima Tajbakhsh, Nirmal Kumar Juluru, Oleg Rybakov, Oleksii Hrinchuk, Oleksii Kuchaiev, Oluwatobi Olabiyi, Pablo Ribalta, Padmavathy Subramanian, Parth Chadha, Pavlo Molchanov, Peter Dykas, Peter Jin, Piotr Bialecki, Piotr Januszewski, Pradeep Thalasta, Prashant Gaikwad, Prasoon Varshney, Pritam Gundecha, Przemek Tredak, Rabeeh Karimi Mahabadi, Rajen Patel, Ran El-Yaniv, Ranjit Rajan, Ria Cheruvu, Rima Shahbazyan, Ritika Borkar, Ritu Gala, Roger Waleffe, Ruoxi Zhang, Russell J. Hewett, Ryan Prenger, Sahil Jain, Samuel Kriman, Sanjeev Satheesh, Saori Kaji, Sarah Yurick, Saurav Muralidharan, Sean Narenthiran, Seonmyeong Bak, Sepehr Sameni, Seungju Han, Shanmugam Ramasamy, Shaona Ghosh, Sharath Turuvekere Sreenivas, Shelby Thomas, Shizhe Diao, Shreya Gopal, Shrimai Prabhumoye, Shubham Toshniwal, Shuoyang Ding, Siddharth Singh, Siddhartha Jain, Somshubra Majumdar, Soumye Singhal, Stefania Alborghetti, Syeda Nahida Akter, Terry Kong, Tim Moon, Tomasz Hliwiak, Tomer Asida, Tony Wang, Tugrul Konuk, Twinkle Vashishth, Tyler Poon, Udi Karpas, Vahid Noroozi, Venkat Srinivasan, Vijay Korthikanti, Vikram Fugro, Vineeth Kalluru, Vitaly Kurin, Vitaly Lavrukhin, Wasi Uddin Ahmad, Wei Du, Wonmin Byeon, Ximing Lu, Xin Dong, Yashaswi Karnati, Yejin Choi, Yian Zhang, Ying Lin, Yonggan Fu, Yoshi Suhara, Zhen Dong, Zhiyu Li, Zhongbo Zhu, and Zijia Chen. Nvidia nemotron nano 2: An accurate and efficient hybrid mamba-transformer reasoning model, 2025. URL https://arxiv.org/abs/2508.14444.

[33] Liping Tang, Nikhil Ranjan, Omkar Pangarkar, Xuezhi Liang, Zhen Wang, Li An, Bhaskar Rao, Linghao Jin, Huijuan Wang, Zhoujun Cheng, Suqi Sun, Cun Mu, Victor Miller, Xuezhe Ma, Yue Peng, Zhengzhong Liu, and Eric P. Xing. Txt360: A top-quality llm pre-training dataset requires the perfect blend, 2024.

[34] Fan Zhou, Zengzhi Wang, Nikhil Ranjan, Zhoujun Cheng, Liping Tang, Guowei He, Zhengzhong Liu, and Eric P Xing. Megamath: Pushing the limits of open math corpora. *arXiv preprint arXiv:2504.02807*, 2025.

[35] Rabeeh Karimi Mahabadi, Sanjeev Satheesh, Shrimai Prabhumoye, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-cc-math: A 133 billion-token-scale high quality math pretraining dataset. *arXiv preprint arXiv:2508.15096*, 2025.

[36] Syeda Nahida Akter, Shrimai Prabhumoye, John Kamalu, Sanjeev Satheesh, Eric Nyberg, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. MIND: Math informed synthetic dialogues for pretraining LLMs. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=TuOTSAiHDn.

[37] Jake Poznanski, Luca Soldaini, and Kyle Lo. olmocr 2: Unit test rewards for document ocr, 2025. URL https://arxiv.org/abs/2510.19817.

[38] Essential AI, :, Andrew Hojel, Michael Pust, Tim Romanski, Yash Vanjani, Ritvik Kapila, Mohit Parmar, Adarsh Chaluvaraju, Alok Tripathy, Anil Thomas, Ashish Tanwer, Darsh J Shah, Ishaan Shah, Karl Stratos, Khoi Nguyen, Kurt Smith, Michael Callahan, Peter Rushton, Philip Monk, Platon Mazarakis, Saad Jamal, Saurabh Srivastava, Somanshu Singla, and Ashish Vaswani. Essential-web v1.0: 24t tokens of organized web data, 2025. URL https://arxiv.org/abs/2506.14111.

[39] OpenAI. gpt-oss-120b gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

[40] Somshubra Majumdar, Vahid Noroozi, Mehrzad Samadi, Sean Narenthiran, Aleksander Ficek, Wasi Uddin Ahmad, Jocelyn Huang, Jagadeesh Balam, and Boris Ginsburg. Genetic instruct: Scaling up synthetic generation of coding instructions for large language models, 2025. URL https://arxiv.org/abs/2407.21077.

[41] Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrapetyan, and Igor Gitman. Openmathinstruct-2: Accelerating ai for math with massive open-source instruction data, 2024. URL https://arxiv.org/abs/2410.01560.

[42] Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, Omri Puny, Ido Galil, Zach Moshe, Tomer Ronen, Najeeb Nabwani, Ido Shahaf, Oren Tropp, Ehud Karpas, Ran Zilberstein, Jiaqi Zeng, Soumye Singhal, Alexander Bukharin, Yian Zhang, Tugrul Konuk, Gerald Shen, Ameya Sunil Mahabaleshwarkar, Bilal Kartal, Yoshi Suhara, Olivier Delalleau, Zijia Chen, Zhilin Wang, David Mosallanezhad, Adi Renduchintala, Haifeng Qian, Dima Rekesh, Fei Jia, Somshubra Majumdar, Vahid Noroozi, Wasi Uddin Ahmad, Sean Narenthiran, Aleksander Ficek, Mehrzad Samadi, Jocelyn Huang, Siddhartha Jain, Igor Gitman, Ivan Moshkov, Wei Du, Shubham Toshniwal, George Armstrong, Branislav Kisacanin, Matvei Novikov, Daria Gitman, Evelina Bakhturina, Jane Polak Scowcroft, John Kamalu, Dan Su, Kezhi Kong, Markus Kliegl, Rabeeh Karimi, Ying Lin, Sanjeev Satheesh, Jupinder Parmar, Pritam Gundecha, Brandon Norick, Joseph Jennings, Shrimai Prabhumoye, Syeda Nahida Akter, Mostofa Patwary, Abhinav Khattar, Deepak Narayanan, Roger Waleffe, Jimmy Zhang, Bor-Yiing Su, Guyue Huang, Terry Kong, Parth Chadha, Sahil Jain, Christine Harvey, Elad Segal, Jining Huang, Sergey Kashirsky, Robert McQueen, Izzy Putterman, George Lam, Arun Venkatesan, Sherry Wu, Vinh Nguyen, Manoj Kilaru, Andrew Wang, Anna Warno, Abhilash Somasamudramath, Sandip Bhaskar, Maka Dong, Nave Assaf, Shahar Mor, Omer Ullman Argov, Scot Junkin, Oleksandr Romanenko, Pedro Larroy, Monika Katariya, Marco Rovinelli, Viji Balas, Nicholas Edelman, Anahita Bhiwandiwalla, Muthu Subramaniam, Smita Ithape, Karthik Ramamoorthy, Yuting Wu, Suguna Varshini Velury, Omri Almog, Joyjit Daw, Denys Fridman, Erick Galinkin, Michael Evans, Katherine Luna, Leon Derczynski, Nikki Pope, Eileen Long, Seth Schneider, Guillermo Siman, Tomasz Grzegorzek, Pablo Ribalta, Monika Katariya, Joey Conway, Trisha Saar, Ann Guan, Krzysztof Pawelec, Shyamala Prayaga, Oleksii Kuchaiev, Boris Ginsburg, Oluwatobi Olabiyi, Kari Briski, Jonathan Cohen, Bryan Catanzaro, Jonah Alben, Yonatan Geifman, Eric Chung, and Chris Alexiuk. Llama-nemotron: Efficient reasoning models, 2025. URL https://arxiv.org/abs/2505.00949.

[43] Run-Ze Fan, Zengzhi Wang, and Pengfei Liu. Megascience: Pushing the frontiers of post-training datasets for science reasoning, 2025. URL https://arxiv.org/abs/2507.16812.

[44] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

[45] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[46] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in neural information processing systems*, 32, 2019.

[47] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[48] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[49] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.

[50] Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. Agieval: A human-centric benchmark for evaluating foundation models, 2023.

[51] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[52] Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.

[53] Jiasheng Zheng, Boxi Cao, Zhengzhao Ma, Ruotong Pan, Hongyu Lin, Yaojie Lu, Xianpei Han, and Le Sun. Beyond correctness: Benchmarking multi-dimensional code generation for large language models, 2024. URL https://arxiv.org/abs/2407.11470.

[54] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.

[55] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*, 2018.

[56] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.

[57] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. 2021.

[58] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=1qvx610Cu7.

[59] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[60] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[61] Qintong Li, Leyang Cui, Xueliang Zhao, Lingpeng Kong, and Wei Bi. Gsm-plus: A comprehensive benchmark for evaluating the robustness of llms as mathematical problem solvers. *arXiv preprint arXiv:2402.19255*, 2024.

[62] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[63] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.

[64] Xinrun Du, Yifan Yao, Kaijing Ma, Bingli Wang, Tianyu Zheng, King Zhu, Minghao Liu, Yiming Liang, Xiaolong Jin, Zhenlin Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025.

[65] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL https://zenodo.org/records/12608602.

[66] Yiwen Hu, Huatong Song, Jia Deng, Jiapeng Wang, Jie Chen, Kun Zhou, Yutao Zhu, Jinhao Jiang, Zican Dong, Wayne Xin Zhao, et al. Yulan-mini: An open data-efficient language model. *arXiv preprint arXiv:2412.17743*, 2024.

A Evaluation Details

A.1 Benchmark Descriptions

This appendix provides detailed descriptions of all 19 benchmarks used in our evaluation, organized by capability domain.

A.1.1 General Knowledge and Reasoning Benchmarks

MMLU (Massive Multitask Language Understanding). MMLU is a comprehensive multiple-choice benchmark covering 57 subjects spanning STEM, humanities, social sciences, and other areas. The dataset tests a model's broad knowledge across diverse domains including elementary mathematics, US history, computer science, law, and more. Questions range from elementary to professional level, providing a thorough assessment of general knowledge and reasoning capabilities.

MMLU-Pro. MMLU-Pro is an enhanced variant of MMLU designed to address the plateau in model performance on the original benchmark. It extends MMLU by: (1) integrating more challenging, reasoning-focused questions; (2) expanding the choice set from four to ten options; and (3) eliminating trivial and noisy questions. MMLU-Pro causes a significant 16-33% accuracy drop compared to MMLU while demonstrating greater stability under varying prompts (2% sensitivity vs. 4-5% in MMLU). The benchmark shows that models utilizing Chain of Thought (CoT) reasoning achieve better performance compared to direct answering.

AGIEval (EN). AGIEval is a human-centric benchmark specifically designed to evaluate foundation models' general abilities in tasks pertinent to human cognition and problem-solving. It is derived from 20 official, public, and high-standard admission and qualification exams, including Chinese College Entrance Exam (Gaokao), American SAT, law school admission tests, math competitions, lawyer qualification tests, and national civil service exams. We evaluate on the English subset (AGIEval-EN) which includes AQUA-RAT, Gaokao English, LogiQA-EN, LSAT (Analytical Reasoning, Logical Reasoning, Reading Comprehension), SAT (English, Math), and MATH problems.

HellaSwag. HellaSwag is a commonsense natural language inference dataset designed to be trivial for humans (>95% accuracy) but challenging for state-of-the-art models. Given an event description (e.g., "A woman sits at a piano"), the task is to select the most likely followup from multiple options. The dataset was constructed via Adversarial Filtering (AF), where discriminators iteratively select adversarial machine-generated wrong answers. The key insight is scaling up the length and complexity of examples to a "Goldilocks" zone where generated text is ridiculous to humans yet often misclassified by models.

TriviaQA. TriviaQA is a large-scale reading comprehension dataset containing over 650K question-answer-evidence triples. It includes 95K question-answer pairs authored by trivia enthusiasts and independently gathered evidence documents (six per question on average) that provide high-quality distant supervision for answering the questions. The dataset tests both reading comprehension and the ability to locate relevant information across multiple documents.

RACE (ReAding Comprehension from Examinations). RACE is a large-scale reading comprehension dataset with more than 28,000 passages and nearly 100,000 questions. The dataset is collected from English examinations in China designed for middle school and high school students. Questions require understanding passage content and reasoning about implicit information, making it a challenging test of reading comprehension capabilities.

WinoGrande. WinoGrande is a collection of 44K problems inspired by the Winograd Schema Challenge, designed to test commonsense reasoning. Formulated as a fill-in-a-blank task with binary options, the goal is to choose the correct option for a given sentence requiring commonsense reasoning. The dataset was adjusted to improve scale and robustness against dataset-specific bias.

OpenBookQA. OpenBookQA is a question-answering dataset modeled after open book exams for assessing human understanding of science. It consists of 5,957 multiple-choice elementary-level science questions (4,957 train, 500 dev, 500 test) that probe understanding of 1,326 core science facts and their application to novel situations. Answering requires both the provided "book" of facts and additional broad common knowledge not contained in the book.

PIQA (Physical Interaction: Question Answering). PIQA is a physical commonsense reasoning benchmark designed to investigate the physical knowledge of language models. The dataset tests reasoning about physical interactions and commonsense understanding of how the physical world works. Questions

are designed to assess to what extent models are actually learning about the physical world versus merely pattern matching.

A.1.2 Code Generation Benchmarks

HumanEval. HumanEval is a code generation benchmark introduced to measure functional correctness for synthesizing Python programs from docstrings. It contains 164 handwritten programming problems with function signatures, docstrings, reference implementations, and multiple unit tests. The benchmark evaluates pass@k metrics, measuring the probability that at least one of k generated samples passes all unit tests. HumanEval has become a standard benchmark for evaluating code generation capabilities of language models.

EvalPlus. EvalPlus extends HumanEval with more comprehensive test suites, providing stricter evaluation than the original benchmark. It augments HumanEval problems with additional test cases that better cover edge cases and corner conditions, making it more difficult to achieve high scores through superficial pattern matching.

MBPP (Mostly Basic Programming Problems). MBPP is a benchmark for program synthesis containing 974 short Python programming tasks designed to be solvable by entry-level programmers. Each problem consists of a natural language description, a reference solution, and three automated test cases. The dataset is designed to measure the ability of models to synthesize short Python programs from natural language descriptions, testing basic programming competence.

A.1.3 Mathematics and STEM Reasoning Benchmarks

GSM8K (Grade School Math 8K). GSM8K is a dataset of 8.5K high-quality, linguistically diverse grade school math word problems requiring multi-step mathematical reasoning. Despite the conceptual simplicity of this problem distribution, even large transformer models struggle to achieve high test performance. The dataset tests whether models can perform the sequential reasoning steps needed to solve elementary mathematics problems. Each question has a natural language solution that demonstrates the reasoning steps.

GSM-Plus. GSM-Plus is a comprehensive benchmark for evaluating the robustness of LLMs as mathematical problem solvers. It extends GSM8K with various mathematical perturbations to test whether models truly understand and apply mathematical knowledge or merely rely on shortcuts. When math questions are slightly changed (new statements added or question targets altered), LLMs can make mistakes even on problems they solved in the original GSM8K, revealing brittleness in their mathematical reasoning.

MATH. MATH is a dataset of 12,500 challenging competition mathematics problems spanning algebra, counting and probability, geometry, intermediate algebra, number theory, prealgebra, and precalculus. Each problem has a full step-by-step solution demonstrating the derivation of the answer. The competition-level difficulty makes MATH one of the most challenging mathematical reasoning benchmarks, requiring sophisticated problem-solving strategies beyond pattern matching.

GPQA (Graduate-Level Google-Proof Q&A). GPQA is a challenging dataset of 448 multiple-choice questions written by domain experts in biology, physics, and chemistry. The questions are designed to be "Google-proof": experts with PhDs reach 65% accuracy (74% when discounting clear mistakes), while highly skilled non-expert validators only reach 3 4% accuracy despite spending over 30 minutes with unrestricted web access. State-of-the-art AI systems also struggle, with GPT-4-based baselines achieving around 39% accuracy. GPQA-Main refers to the main variant of this benchmark.

SuperGPQA. SuperGPQA is an extended variant of GPQA with additional challenging questions in biology, physics, and chemistry. It maintains the graduate-level difficulty and Google-proof property of the original GPQA while expanding coverage of advanced scientific topics.

MMLU-STEM. This is the STEM-focused subset of MMLU, including subjects such as abstract algebra, astronomy, college biology, college chemistry, college computer science, college mathematics, college physics, computer security, elementary mathematics, high school biology, high school chemistry, high school computer science, high school mathematics, high school physics, high school statistics, machine learning, and electrical engineering.

MMLU-Pro-STEM. This is the STEM subset of MMLU-Pro, maintaining the increased difficulty and expanded choice set (10 options) of MMLU-Pro while focusing specifically on STEM disciplines. It provides an even more challenging test of scientific and mathematical knowledge than MMLU-STEM.

Table 10: Impact of learning rate decay strategy on Stage 2-2 performance. Constant LR maintains 3e-5 throughout, while cosine decay gradually reduces LR from 3e-5 to 3e-6.

Configuration	Avg General	Avg Code	Avg Science	Overall Avg
Constant LR	52.06	52.32	45.44	49.66
Cosine decay	**52.49**	**53.31**	**45.39**	**50.00**
Δ (decay - constant)	+0.43	+0.99	-0.05	+0.34

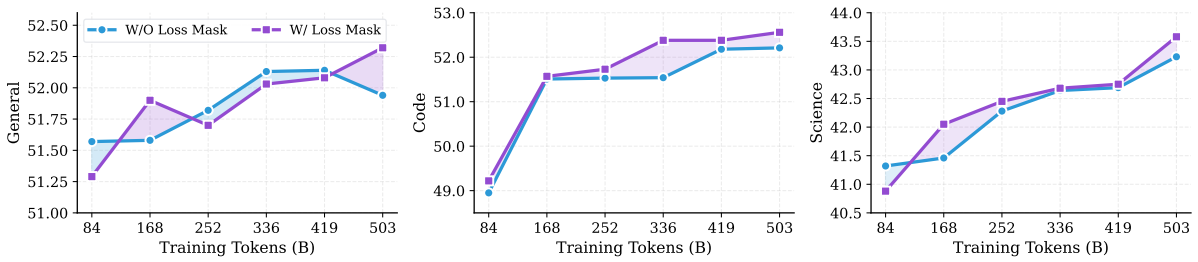


Figure 13: Impact of QA masking policy on three capability dimensions across training steps in Stage 2-2. The comparison between masked and unmasked configurations shows marginal improvements, with consistent but modest gains across all capability clusters.

B Training Implementation Decisions: LR Decay and QA Masking

Beyond data mixture design, two practical implementation choices can meaningfully influence training outcomes yet are rarely studied in pretraining contexts: **learning rate decay scheduling** and **QA question masking policy**. LR decay schedules determine how aggressively the model consolidates knowledge in later training steps, while masking policy governs whether the model is trained to predict question tokens or only answer tokens. Although both choices may seem like minor engineering details, their interaction with the data format and training stage can lead to non-trivial performance differences. We systematically investigate both decisions to provide principled guidance for Stage 2 configurations.

Learning rate decay. We compare constant LR against cosine decay (decreasing from 3e-5 to 3e-6) for the 70% QA Stage 2-2 configuration (Table 10). Cosine decay yields consistent improvements in general knowledge and code generation, while science reasoning remains largely unchanged. The benefits align with a "capability building then refinement" pattern: aggressive learning in Stage 2-1 establishes broad capabilities, and a gradual LR reduction in Stage 2-2 consolidates and stabilizes them. The overall improvement is modest but reliable, making cosine decay a low-cost enhancement worth applying in multi-substage configurations.

QA data masking policy. We compare masking the question portion of QA pairs (only answer tokens contribute to loss, following SFT convention) against treating QA as continuous text (all tokens contribute). Using the 70% QA Stage 2-2 configuration with high-quality, diverse question data, masking achieves 49.14 overall at 30k steps versus 48.77 without masking (+0.37). This marginal gain is consistent across general (+0.38), code (+0.35), and science (+0.35) domains, and training dynamics remain stable under both strategies. In contrast to SFT—where masking is essential to prevent shortcut learning—pretraining's broader knowledge acquisition objective means that models benefit from learning *both* question understanding and answer generation when questions are sufficiently diverse. Practically, masking incurs non-trivial engineering overhead (additional mask storage or runtime computation) for a modest return; the key takeaway is that **question quality and diversity matter more than masking strategy**. Figure 13 illustrates the comparison across training checkpoints.

C Prompts for Dataset Construction

Rewriting Prompt for Constructing MegaMath-Web-Pro

```
Task:
- Carefully analyze the provided text to extract key facts, concrete details,
important numbers, and core concepts.
- Remove any irrelevant or noisy information, and reorganize the content into
a
logically structured, information-dense, and concise version that is easy to
learn from. Output only the refined text.
- Strive to maintain the original length as much as possible (avoid excessive
shortening).
Text:
<EXAMPLE>
Just output the refined text, no other text.
```

Darwin-Science L4 Processing Prompt

```
You are an expert document cleaner specialized in identifying and removing
unwanted content and correcting OCR errors from various document (mainly
academic) chunks.

## Objective:
Clean and standardize OCR text by identifying and removing redundant,
erroneous, or unwanted content and correcting obvious OCR errors according to
the rules below. Your task is to identify and delete unnecessary content
completely, fix technical errors, while preserving all academic value.

## Deletion and Correction Rules:
### Document Structural Deletion
* Remove **table of contents and navigation structures**: Multiple
consecutive chapter/section titles listed together without accompanying text
content
  - **Preserve content section headings in main text**: such as chapter
  headings, section titles followed by explanatory text or academic material
* Remove **reference lists completely**: numbered entries with author names,
publication titles, and years (e.g., "1. Smith, J. (2020). Title. Journal,
15(3), 123-145.") **[Delete entire list regardless of format]**
* Remove **front matter and back matter**: such as prefaces, acknowledgments,
copyright statements, indexes, and other standard book structural elements
  - **Preserve sections with academic value**: such as abstracts,
  introductions, conclusions that present research background or methodology
* Remove **publication and metadata information**: such as ISBN, publisher
information, revision history, version numbers, institutional affiliations,
author affiliations, addresses, contact information
* Remove **page headers, page footers, and page numbers**

### Academic Content Deletion
* Remove **pure indexing appendices**: such as glossaries, symbol tables,
abbreviation lists, indexes, notations and other purely referential lookup
content (entries that only provide definitions without explanations, e.g., "a
- alpha coefficient")
  - **Preserve**: appendices with learning value (e.g. mathematical
  derivations, proofs, technical explanations)
  - **Preserve**: explanatory content that directly supports main text
  elements (e.g. abbreviation/parameter explanations after
  tables/formulas/diagrams)
* Remove **image files and placeholders**: such as `<img>` tags, image file
paths, image URLs, markdown syntax and image placeholders (e.g. `[Image]`,
`[Picture not available]`)
  - **Preserve**: figure/table titles, descriptive text (including content
  within markdown image formats: ![description](path) → description)
  - **Preserve**: in-text references (e.g., "as shown in Figure 1")

### Invalid and Redundant Content Deletion
```

```
* Remove **OCR processing artifacts**: such as garbled text, encoding
artifacts, duplicate characters, malformed special characters, OCR messages
(`[OCR error]`), file paths, timestamps, version numbers, revision history
* Remove **garbage content**: such as junk information, advertising content,
placeholders (e.g. [Insert citation here])
* Remove **duplicate content**: identical paragraphs or sections mainly
caused by OCR errors
  - **Exception**: Carefully apply to technical formulas, equations, or
  specialized notation that may contain subtle but meaningful differences
  - **Exception**: Apply contextual analysis - preserve identical content
  that serves different semantic purposes or artistic purposes (e.g., poetic
  refrains, literary repetition)
* Remove **content and navigation markers**: [content missing], [page break],
(Continued), and similar placeholder markers
* Remove **URLs and links**: all web addresses, hyperlinks, and link
information

### OCR Error Correction
* **Fix text fragmentation**: repair split words, broken sentences, erroneous
line breaks and paragraph divisions, missing spaces and punctuation
* **Fix fragmented structured content**: Repair OCR-damaged structured
content (e.g. tables, diagrams, formulas) appearing as consecutive lines of
isolated words, single characters, or short phrases
  - **Pattern**: Consecutive lines (5+) with 1-3 words/characters each
  - **Action**: Preserve content while indicating structural damage; delete
  if unrepairable
* **Standardize whitespace and formatting**: clean excessive whitespace,
compress blank lines, standardize spacing and indentation
* **Fix character and encoding errors**: correct obvious character errors,
spelling issues, and Unicode anomalies
* **Standardize punctuation**: unify quotation marks, dashes, hyphens, and
other punctuation
* **Complete truncated words**: only fix obviously incomplete words from
clear OCR errors, avoid modifying content at chunk edges
* **Standardize academic formatting**: remove excessive LaTeX commands and
unify notation format

## Content Protection Rules:
### Always Preserve Academic and Educational Content
* Preserve **Technical and specialized content**: such as formulas,
equations, proofs, symbols, chemical structures, biological sequences and
their original format
  - **Preserve exact content**: do not alter variables, coefficients,
  structures, sequences, or any technical details
* Preserve **In-text references and citations**: such as (Smith, 2020), [15],
"see Chapter 2", equation (5), "Figure 2.5", (pp. 3-7)
* Preserve **Table structures**: preserve academic table content, formatting
and structural markers (e.g. "|", HTML tags)
  - **Exception**: Does not apply to navigation tables (table of contents,
  indexes, glossaries) which should be removed
* Preserve **Code blocks and programming examples**: preserve code block
markers (```language, ```, etc.) and internal code syntax and structure
* Preserve **Educational content**: such as exercises, questions, answers,
solutions, case studies, instructions, user guides
* Preserve **Explanatory content**: such as NOTE boxes, WARNING boxes, tips,
author comments, supplementary information, academic footnotes
* Preserve **Chunk boundary content**: incomplete sentences and words at
chunk edges due to text segmentation
* Preserve **Literary and humanities content**: including poetry, fiction,
drama, creative writing, literary analysis, philosophical texts, and other
humanities scholarship with educational value

## Instructions:
- Carefully identify all content matching the deletion rules
- Remove completely any content that should be deleted
- Preserve all valuable academic content by applying protection rules and
retaining content that doesn't match deletion rules
- Apply OCR error corrections to fix obvious technical problems
```

```
- Ensure text flows naturally after corrections and deletions
- If the entire chunk should be deleted, leave the output tags completely
empty
- **Important**: The content inside the <CLEANED_TEXT> tags must be exactly
the text after deletion, with no explanations, comments, or additional text
inside the tags

## Input:
OCR document chunk:
[CHUNK]

## Output Format:
<CLEANED_TEXT>
[Place the cleaned content here, or leave completely empty if everything
should be deleted]
</CLEANED_TEXT>
```

Darwin-Science L5 Processing Prompt

```
You are a master science communicator and pedagogical expert. Your mission is
to transform the following dense, expert-level text chunk into vibrant,
crystal-clear educational material. Imagine you are creating a definitive
learning resource for a bright but novice audience. Your goal is not merely
to simplify, but to deeply elucidate, making the complex intuitive and the
implicit explicit.

Your transformation will be governed by two sets of principles: the Core
Mandate (what you must actively do) and the Unbreakable Rules (what you must
never violate).

### The Unbreakable Rules: Fidelity and Integrity
This principle is of paramount importance and must be followed without
exception to ensure the output is valid.

* **(a) Scientific and Factual Correctness:** Maintain absolute rigor. All
data, formulas, definitions, theories, experimental results, and logical
arguments must be preserved without altering their meaning or context. Your
additions must clarify, not contradict.

* **(b) Structural Integrity:** Preserve the original structure flawlessly.
Keep ALL section headers (`##`, `###`), figure/table labels, equation numbers,
etc., exactly as they appear, especially at the beginning and end of the
chunk.

* **(c) Contextual Limitation and Termination:** You are processing a partial
*chunk* of a document. You lack the full context. Therefore, you must work
**strictly** within the provided text. Do not invent definitions or reference
goals from outside the chunk. **This strict adherence means your output must
terminate exactly where the provided chunk terminates.** If the chunk ends
abruptly (e.g., at a new section header, in the middle of a sentence, or with
a label), your output **must be cut off at that exact same point.** This is
the single most critical rule for preventing hallucination and ensuring
continuity.

### The Core Mandate: Deep Pedagogical Transformation
This is your primary objective. Be bold and proactive in adding educational
value. Your goal is to weave a rich tapestry of understanding.
```

```
* **(a) Deconstruct and Narrate the 'Why':** This is your primary mode of
explanation. Actively expand on logical leaps. When the text says "it follows
that," "clearly," or "trivially," you must step in and meticulously detail
the intermediate steps. More importantly, you must articulate the expert's
internal monologue. When faced with an equation, a problem, or a logical
step, explain the strategy. Ask and answer questions like: "Okay, what's our
goal here?" "What's the first thing I should look for when I see an equation
like this?" "We're going to use technique X, and here's why it's the right
tool for this specific job." Your mission is to reveal the problem-solving
journey, making every single connection transparent.

* **(b) From Jargon to Insight:** When you encounter a crucial technical
term, or a significant variable within a formula, you must deliberately pause
the narrative to explain it. Don't just provide a dry definition. Elucidate
its importance: What role does this term or variable play? Why does it
matter? Crucially, you must then use simpler language, vivid analogies, or
concrete examples to build a strong and intuitive mental model for the reader
before you continue with the main explanation. This ensures no reader is left
behind due to unfamiliar notation or terminology.

* **(c) Invent Vivid Analogies and Concrete Examples:** Go beyond the text.
Where a concept is abstract, create a simple, concrete example to illustrate
it. Invent memorable analogies that connect the new information to a
learner's existing knowledge (e.g., electron shells as floors in a hotel).

* **(d) Create Contextual Bridges:** Weave a narrative thread by connecting
the current idea to the broader field of knowledge. Hint at future
applications or link back to more foundational concepts. For instance: "This
principle of [X] is a cornerstone of the field and will be essential for
understanding [Y] later on."

* **(e) Think Like a Learner:** Proactively identify points of potential
confusion. What questions would a curious student ask here? Answer them
before they are asked. A great teacher warns students about common mistakes.
Where applicable, insert brief, helpful asides that feel like a mentor's
margin notes.

* **(f) Prioritize Narrative Flow and Clean Formatting:** When you encounter
messy or noisy original LaTeX formatting, convert it to a clean and pristine
style (especially for formulas and tables). Above all, strive for a smooth,
cohesive, and engaging narrative. Your writing should feel like a continuous,
guided tour through the material, not a collection of disconnected facts and
callouts. To that end, you must avoid the overuse of overly-structured,
point-by-point expressions. Let the main text flow logically and tell a
story, adopting the persona of an extremely patient and encouraging teacher.

---
Summary of Principles Above: To sum it up, you must strictly respect the
accuracy and structure of the original chunk while doing everything possible
to make the rewritten text easier to learn and to lower the reader's
cognitive load. Consequently, the rewritten text will typically be more
detailed and thus longer than the original.
---

**Crucial Output Instructions:**
1.  **Self-Contained Output:** The refined text must stand on its own. Avoid
any meta-commentary or phrases that refer to the original text, such as "the
original paper," "the original context," "the original chunk", etc. The goal
is to create a seamless, self-contained educational text, not a commentary on
another document.
2.  **Strict Termination:** You **MUST** terminate your output at the
**EXACT** same point the provided chunk terminates. Do not write a single
character past the end of the original chunk. In particular, if a chunk ends
with the start of a new section, subsection, step (e.g., it starts with a
heading) or cuts off in the middle of a proof/solution, you must NOT invent
or continue writing ANY content that would follow.**

---
```

```
*You must output ONLY the refined chunk itself, without any introductory or
concluding remarks.*

**Original text:**
{chunk}

**Refined text:**
```

QA Extraction Prompt for Darwin-Science-Book-QA(Biology)

```
Below is a document extract from biological sciences domain (biology,
ecology, medicine, genetics, physiology, etc.).

# Extraction Task
Extract complete, independently solvable biology Q&A pairs following these
strict guidelines:

## CRITICAL RULES (Must Follow):
1. **Answer-First Principle**: Only extract a question if its complete answer
exists in the document
2. **Zero References Requirement**:
   - Questions: No references to "the document", "the text", "Figure 1",
   "Table 2", "above", "below", "the experiment above","as discussed in the
   extract"etc.
   - Answers: No references to "as in document", "as shown in text", "see the
   document", "Table 2", etc.
   - Replace ALL references with actual content from the document
3. **Complete Independence**:
   - Each question must be self-contained with ALL necessary information, as
   others will not have access to the original text.
   - If a question is incomplete, incorporate missing context to make it
   fully solvable
   - Multiple questions must be mutually independent (no "as in first
   Question", "using previous result")
4. **Answer Must Exist**: All answers must be directly found or clearly
derivable from the given text - do not extract if uncertain
5. **Value-Based Selection**: Focus on extracting the MOST IMPORTANT and
EDUCATIONAL content
6. **No Data-Dependent Conclusions**: Do NOT extract questions whose answers
rely heavily on complex data tables or extensive numerical data that cannot
be reasonably included in the Q&A format


## For Questions/Problems:
- **Extract explicit question-answer pairs** if they exist (ensure they meet
zero references and independence requirements)
- **For Mechanisms/Processes**: Create questions about how biological
processes work (e.g., "How does X process occur?", "Describe the mechanism
of...")
- **For Structure-Function**: Create questions linking structure to function
(e.g., "What is the structure and function of X?")
- **For Phenomena and Causes**: Create questions exploring why biological
phenomena occur (e.g., "Why does X happen?", "What causes X?")
- **For Comparisons**: Create questions comparing biological entities (e.g.,
"What are the differences between X and Y?", "How does X differ from Y?")
- **For Diseases**: Create questions about etiology, symptoms, mechanisms, or
treatments (when information is complete)
- **For Experiments**: Extract questions about experimental design, findings,
or significance (only if self-contained without heavy data table dependence)
- **For Evolution/Ecology**: Create questions about evolutionary
explanations, adaptations, or ecological relationships
- **For Concepts/Definitions**: Ask about them flexibly, but focus on core
concepts and ensure complete answers exist
- **Critical**: Each question must be complete and self-contained, as others
will not have access to the original text.
- **Do NOT extract**:
```

```
  - Questions that require additional data to be provided in order to be
  answered
  - Questions whose answers are uncertain or incomplete
- If no valid Q&A content can be extracted following above rules, return
exactly: [NO QA]

## For Answers:
- **Process**: First locate the complete answer in the document, then
organize it clearly
- If the answer has explanation, reorganize it into a clear, well-structured
format
- If the answer lacks explanation, add necessary biological reasoning or
context
- For final answers that need exact matching (multiple-choice, calculations,
fill-in-the-blank, true/false), use $
boxedanswer$ notation

## Output Format (STRICT):
Use this exact format for parsing:

---BEGIN QA---
Question: <First complete question>
Answer: <First complete answer>
---END QA---

---BEGIN QA---
Question: <Second complete question>
Answer: <Second complete answer>
---END QA---

If no valid Q&A can be extracted, return exactly: [NO QA]

---
Extract:
chunk

Now process the extract and return the result in the specified format.
```

QA Extraction Prompt for Darwin-Science-Book-QA(Chemistry)

```
Below is a document extract from chemistry domain.

# Extraction Task
Extract complete, independently solvable chemistry Q&A pairs following these
strict guidelines:

## CRITICAL RULES (Must Follow):
1. **Answer-First Principle**: Only extract a question if its complete answer
exists in the document
2. **Zero References Requirement**:
   - Questions: No references to "the document", "the text", "Figure 1",
   "Scheme 45", "compound 201", "above", "below", etc.
   - Answers: No references to "as in document", "as shown in text", "see
   Scheme", "compound 201", etc.
   - Replace ALL references with actual content (e.g., replace "compound 201"
   with its chemical name)
3. **Complete Independence**:
   - Each question must be self-contained with ALL necessary information, as
   others will not have access to the original text.
   - If a question is incomplete, incorporate missing context to make it
   fully solvable
   - For multi-step reactions, include all necessary steps and conditions
   - Multiple questions must be mutually independent (no "as in first
   Question", "using previous result")
```

```
4. **Answer Must Exist**: All answers must be directly found or clearly
derivable from the given text - do not extract if uncertain
5. **No Scheme/Figure-Dependent Content**: Do NOT extract questions whose
answers rely heavily on reaction schemes, structural diagrams, or figures
that cannot be described adequately in text

## For Questions/Problems:
- Extract explicit chemistry questions ONLY if their complete answers are
present
- **Prioritize substantive content** over trivial details: focus on chemical
properties, principles, mechanisms, experimental methods, calculations, and
applications rather than isolated nomenclature or data-dependent conclusions
- **For Chemical Properties**: Create questions about physical/chemical
properties, reactivity, or characteristics when well-described
- **For Chemical Principles/Theories**: ask to explain the principle or test
understanding of key concepts (e.g., "Explain Le Chatelier's principle",
"What happens to equilibrium when...")
- **For Chemical Equations**: ask about equation balancing, reaction types,
or explain the reaction principles (include complete conditions)
- **For Reaction Mechanisms**: Create questions about how reactions proceed
ONLY if describable in text without complex schemes
- **For Chemical Calculations**: Extract calculation problems with ALL given
values, units, and formulas only if the answer exits in the text
- **For Experiments**: create questions about experimental methods,
observations, or interpret/explain phenomena (include complete procedural
steps)
- **Critical**: Each question must be complete and self-contained
- **Do NOT extract**:
  - Questions depending on reaction schemes or structural diagrams not
  describable in text
  - Questions requiring extensive analytical data tables or spectroscopic
  data
  - Questions that require additional data to be provided in order to be
  answered
  - Questions whose answers are uncertain or incomplete
  - Questions whose answers can't be found in the text
- If no valid Q&A content can be extracted following above rules, return
exactly: [NO QA]

## For Answers:
- **Process**: First locate the complete answer in the document, then
organize it clearly
- For numerical answers: include units and appropriate significant figures
- If the answer has explanation, reorganize it into a clear, well-structured
format
- If the answer lacks explanation, add necessary context or reasoning
- For final answers that need exact matching (multiple-choice, calculations,
fill-in-the-blank, true/false), use $
boxedanswer$ notation

## Output Format (STRICT):
Use this exact format for parsing:

---BEGIN QA---
Question: <First complete question>
Answer: <First complete answer>
---END QA---

---BEGIN QA---
Question: <Second complete question>
Answer: <Second complete answer>
---END QA---

If no valid Q&A can be extracted, return exactly: [NO QA]

---
Extract:
chunk
```

```
Now process the extract and return the result in the specified format.
```

QA Extraction Prompt for Darwin-Science-Book-QA(Computer Science)

```
Below is a document extract from computer science domain (programming,
systems, algorithms, software engineering, etc.).

# Extraction Task
Extract complete, independently solvable CS Q&A pairs following these strict
guidelines:

## CRITICAL RULES (Must Follow):
1. **Answer-First Principle**: Only extract a question if its complete answer
exists in the document
2. **Zero References Requirement**:
   - Questions: No references to "the document", "the text", "Figure 1",
   "Section 2.1", "above", "below", "the code above","in Algorithm 4.2" etc.
   - Answers: No references to "as in document", "as shown in text", "see the
   document", "Section 2.1", etc.
   - Replace ALL references with actual content from the document
3. **Complete Independence**:
   - Each question must be self-contained with ALL necessary information, as
   others will not have access to the original text.
   - If a question is incomplete, incorporate missing context to make it
   fully solvable
   - For code-related questions, include necessary code context within the
   question
   - Multiple questions must be mutually independent (no "as in first
   Question", "using previous result")
4. **Answer Must Exist**: All answers must be directly found or clearly
derivable from the given text - do not extract if uncertain
5. **Value-Based Selection**: Focus on extracting the MOST IMPORTANT and
EDUCATIONAL content


## For Questions/Problems:
- Extract explicit questions ONLY if their complete answers are present
- **For Step-by-Step Procedures**: Extract as "How to..." questions ONLY if
ALL steps are present and self-contained
- **For System Design/Architecture**: Create questions about design
decisions, trade-offs, or evolution (e.g., "Why did BGS migrate from Mimer to
Oracle?")
- **For Algorithms/Code**: Questions can focus on functionality, complexity,
implementation details, or usage, if including code, ensure sufficient
context is provided in the question
- **For Comparisons**: Create questions like "What are the differences
between X and Y?" or "When to use X vs Y?"
- **For Performance**: Extract questions about optimization techniques,
performance metrics, or bottlenecks
- **For Concepts/Definitions**: Ask about them flexibly, but ensure complete
answer exists and focus on core concepts
- **Formulate questions naturally and appropriately** - use varied question
formats based on content type, but make sure the answer can be found in the
document
- If no valid Q&A content can be extracted following above rules, return
exactly: [NO QA]

## For Answers:
- **Process**: First locate the complete answer in the document, then
organize it clearly
- If the answer has explanation, reorganize it into a clear, well-structured
format
- If the answer lacks explanation, add necessary context or reasoning
- For final answers that need exact matching (multiple-choice, calculations,
fill-in-the-blank, true/false), use $
boxedanswer$ notation
```

```
## Output Format (STRICT):
Use this exact format for parsing:

---BEGIN QA---
Question: <First complete question>
Answer: <First complete answer>
---END QA---

---BEGIN QA---
Question: <Second complete question>
Answer: <Second complete answer>
---END QA---

If no valid Q&A can be extracted, return exactly: [NO QA]

---
Extract:
chunk

Now process the extract and return the result in the specified format.
```

QA Extraction Prompt for Darwin-Science-Book-QA(Engineer)

```
Below is a book document extract.

# Extraction Task
Extract complete, independently solvable engineering questions and answers
from the document while following these guidelines:

## For Questions:
- Extract any explicit engineering questions with their associated answers
- For implicit engineering principles, design methods, procedures, or
technical specifications presented as statements, convert them to well-formed
questions ONLY if they can stand alone
- Ensure each extracted question contains ALL necessary information to be
solved independently without requiring additional context
- Include any relevant diagrams, circuit schematics, system designs,
specifications, or technical data mentioned (describe them if not visible)
- Extract multiple questions separately if they exist
- **If a question lacks necessary context (specifications, constraints,
parameters, standards, initial conditions), incorporate the missing
information from the document to make it self-contained**
- If no engineering content can be meaningfully extracted as a question,
return `[NO QA]`

## For Answers:
- Include the answer provided in the extract
- Answers should capture the essential explanation of the engineering
principle, design approach, or solution methodology
- If the source material contains design procedures, calculation steps, or
implementation methods, include these in the answer
- For engineering problems, the answer should explain the approach,
assumptions, calculations, and practical considerations as presented in the
text
- If the answer already has explanation, reorganize the solution into a clear
and well-structured format for better readability and understanding
- If the answer lacks explanation, add necessary intermediate steps,
reasoning, and justifications as a teacher would
- For final answers that need exact matching (calculations, design values,
multiple-choice, true/false), use $
boxed$ notation
- Include units, dimensions, and tolerances where applicable

## Requirements:
- The question should include all necessary information (specifications,
constraints, parameters, standards)
```

```
- The answer should be practical, accurate, and well-explained
- Both question and answer should stand alone (no references to documents or
original materials)
- Preserve technical terminology, symbols, and notation accurately
- Maintain engineering rigor and attention to practical feasibility
- Include safety considerations and industry standards where relevant

## Format:
Format each question-answer pair as:
Question: [Complete engineering question with all context needed to
understand]
Answer: [Corresponding answer from the text]

The extract is as follows:
chunk

Now process the extract and return the result.
```

QA Extraction Prompt for Darwin-Science-Book-QA(Humansocial)

```
Below is a document extract from humanities and social sciences domain.

# Extraction Task
Extract complete, independently solvable humanities/social sciences Q&A pairs
following these strict guidelines:

## CRITICAL RULES (Must Follow):
1. **Answer-First Principle**: Only extract a question if its complete answer
exists in the document
2. **Zero References Requirement**:
   - Questions: No references to "the document", "the text", "the author",
   "Chapter 1", "above", "below", etc.
   - Answers: No references to "as in document", "as shown in text", "the
   author states", etc.
   - Replace ALL references with actual content from the document
3. **Complete Independence**:
   - Each question must be self-contained with ALL necessary information
   - If a question is incomplete, incorporate missing context to make it
   fully solvable
   - Multiple questions must be mutually independent (no "as in first
   Question", "using previous result")
4. **Answer Must Exist**: All answers must be directly found or clearly
derivable from the given text - do not extract if uncertain
5. **Value-Based Selection**: Focus on extracting the MOST IMPORTANT and
EDUCATIONAL content with analytical substance
6. **Knowledge-Based Content Only**: Only extract from texts that convey
factual information, scholarly theories, analytical arguments, or objective
descriptions. Do NOT extract from:
   - Narrative fiction, creative writing, or purely storytelling passages
   (novels, literary narratives)
   - Religious proselytizing, propaganda, or ideological indoctrination
   - Advertising, marketing, or promotional content
   - Pure emotional expressions, personal rants, or subjective opinions
   without analytical substance
   - Inflammatory or extremist rhetoric

## For Questions/Problems:
- **Extract explicit question-answer pairs** if they exist (ensure zero
references and independence requirements)
- **Create questions about**:
  - Historical events: causes, consequences, significance, key figures
  - Theories and concepts: definitions, frameworks, applications,
  explanations
  - Arguments and viewpoints: main claims, evidence, reasoning, limitations
  - Comparisons: differences and similarities between
  theories/events/policies/concepts
```

```
  - Impacts and significance: effects, influence, historical/social
  importance
- **Formulate questions naturally** - use varied formats based on content
type
- **Critical**: Each question must be self-contained with necessary context
and clearly distinguish facts from interpretations
- **Do NOT extract**:
  - Questions from narrative fiction, religious proselytizing, propaganda,
  advertising, or purely emotional content
  - Rhetorical questions without answers in the text
  - Open-ended questions without clear answers
  - Questions requiring extensive background not provided
  - Trivial details without analytical significance
- If no valid Q&A content can be extracted following above rules, return
exactly: [NO QA]

## For Answers:
- **Process**: First locate the complete answer in the document, then
organize it clearly
- If the answer has explanation, reorganize it into a clear, well-structured
format
- If the answer lacks explanation, add necessary contextual reasoning while
staying true to the text
- For final answers that need exact matching (multiple-choice, calculations,
fill-in-the-blank, true/false), use $
boxed$ notation

## Output Format (STRICT):
Use this exact format for parsing:

---BEGIN QA---
Question: <First complete question>
Answer: <First complete answer>
---END QA---

---BEGIN QA---
Question: <Second complete question>
Answer: <Second complete answer>
---END QA---

If no valid Q&A can be extracted, return exactly: [NO QA]

---
Extract:
chunk

Now process the extract and return the result in the specified format.
```

QA Extraction Prompt for Darwin-Science-Book-QA(Math)

```
# Extraction Task
Extract complete, independently solvable mathematical Q&A pairs following
these strict guidelines:

## CRITICAL RULES (Must Follow):
1. **Answer-First Principle**: Only extract a question if its complete answer
exists in the document
2. **Zero References Requirement**:
   - Questions: No references to "the document", "the text", "Figure 1",
   "Theorem 1.1", "above", "below", etc.
   - Answers: No references to "as in document", "as shown in text", "see the
   document","by Theorem 1.1" etc.
   - Replace ALL references with actual content from the document
3. **Complete Independence**:
   - Each question must be self-contained with ALL necessary information
   - If a question is incomplete, incorporate missing context to make it
   fully solvable
```

```
   - Multiple questions must be mutually independent (no "as in first
   Question ", "using previous result")
4. **Answer Must Exist**: All answers must be directly found or clearly
derivable from the given text - do not extract if uncertain

## For Questions/Problems:
- Extract explicit mathematical questions ONLY if their complete answers are
present
- For **Theorem/Proposition/Corollary + Proof/DEMONSTRATION** pairs: Convert
into "Prove that [full theorem statement]" format
- For **Standalone Theorems** (without proof): Create questions about the
theorem content (e.g., if theorem states "(x-y)(x+y)=x²-y²", ask
"(x-y)(x+y)=?" or ask to state/explain the theorem) ONLY if the answer
content is in the text
- For **Concepts/Definitions**: You may ask about them, but ensure the
complete answer can be found in the document
- **If you see only solution steps without a question, DO NOT create a
question**
- If no valid Q&A content can be extracted following above rules, return
exactly: [NO QA]

## For Answers:
- **Process**: First locate the complete answer in the document, then enhance
the reasoning steps
- Include the provided solution, proof, or demonstration when available
- **Replace all references**: Do NOT use "by Theorem 1.2", "using Lemma 3",
"from Definition 2.1", etc. Instead, state the actual
theorem/lemma/definition content or rephrase the reasoning without references
- For theorems/propositions, the answer should contain the complete proof
- If the answer has explanation, reorganize it into a clear, well-structured
format
- If the answer lacks explanation, add necessary intermediate reasoning as a
teacher would
- For final answers that need exact matching (multiple-choice, calculations,
fill-in-the-blank, true/false), use $
boxedanswer$ notation

## Output Format (STRICT):
Use this exact format for parsing:

---BEGIN QA---
Question: <First complete question>
Answer: <First complete answer>
---END QA---

---BEGIN QA---
Question: <Second complete question>
Answer: <Second complete answer>
---END QA---

If no valid Q&A can be extracted, return exactly: [NO QA]


Extract:
chunk

Now process the extract and return the result in the specified format.
```

QA Extraction Prompt for Darwin-Science-Book-QA(Medicine)

```
Below is a document extract from medical sciences domain (clinical medicine,
pharmacology, physiology, pathology, public health, etc.).

# Extraction Task
Extract complete, independently solvable medical Q&A pairs following these
strict guidelines:
```

```
## CRITICAL RULES (Must Follow):
1. **Answer-First Principle**: Only extract a question if its complete answer
exists in the document
2. **Zero References Requirement**:
   - Questions: No references to "the document", "according to the text",
   "Figure 1", "Table 3", "above", "below", "the study above", etc.
   - Answers: No references to "as in document", "as shown in text", "see
   Table", "Figure 1", etc.
   - Replace ALL references with actual content from the document
3. **Complete Independence**:
   - Each question must be self-contained with ALL necessary information
   - If a question is incomplete, incorporate missing context to make it
   fully solvable
   - Multiple questions must be mutually independent (no "as in first
   Question", "using previous result")
4. **Answer Must Exist**: All answers must be directly found or clearly
derivable from the given text - do not extract if uncertain
5. **Value-Based Selection**: Focus on extracting the MOST IMPORTANT and
EDUCATIONAL medical content
6. **No Data-Dependent Conclusions**: Do NOT extract questions whose answers
rely heavily on complex data tables, diagnostic images, or clinical charts
that cannot be adequately described in text

## For Questions/Problems:
- **Extract explicit question-answer pairs** if they exist (ensure zero
references and independence requirements)
- **Prioritize substantive clinical and medical content** over trivial
details or data-dependent conclusions
- **For Disease-related content**: Create questions about definition,
classification, pathophysiology, clinical manifestations, diagnosis,
prevention, complications, or risk factors
- **For Drug/Treatment-related content**: Create questions about drug
composition, indications, mechanisms of action, dosage, side effects, or
other treatment modalities
- **For Physiological mechanisms**: Create questions about normal body
functions, pathological processes, biochemical mechanisms, or metabolic
pathways
- **For Concepts and comparisons**: Create questions about medical
definitions, comparisons between diseases/treatments/approaches, or
differential diagnosis
- **Critical**: Each question must be complete and self-contained, as others
will not have access to the original text.
- **Do NOT extract**:
  - Questions depending on diagnostic images, complex clinical charts, or
  detailed lab data tables
  - Questions requiring extensive clinical case details not fully provided
  - Questions whose answers are uncertain, speculative without clear
  indication, or incomplete
  - Medical advice for specific individual cases (focus on general medical
  knowledge)
- If no valid Q&A content can be extracted following above rules, return
exactly: [NO QA]

## For Answers:
- **Process**: First locate the complete answer in the document, then
organize it clearly
- If the answer has explanation, reorganize it into a clear, well-structured
format
- If the answer lacks explanation, add necessary medical reasoning or context
- For final answers that need exact matching (multiple-choice, calculations,
fill-in-the-blank, true/false), use $
boxed$ notation

## Output Format (STRICT):
Use this exact format for parsing:

---BEGIN QA---
Question: <First complete question>
```

```
Answer: <First complete answer>
---END QA---

---BEGIN QA---
Question: <Second complete question>
Answer: <Second complete answer>
---END QA---

If no valid Q&A can be extracted, return exactly: [NO QA]

---
Extract:
chunk

Now process the extract and return the result in the specified format.
```

QA Extraction Prompt for Darwin-Science-Book-QA(Physics)

```
Below is a book document extract.

# Extraction Task
Extract complete, independently solvable physics content following these
guidelines:

## CRITICAL RULES (Must Follow):
1. **Answer-First Principle**: Only extract a question if its complete answer
exists in the document
2. **Zero References Requirement**:
   - Questions: No references to "the document", "the text", "Figure 1",
   "Theorem 1.1", "above", "below", etc.
   - Answers: No references to "as in document", "as shown in text", "see the
   document","Theorem 1.1" etc.
   - Replace ALL references with actual content from the document
3. **Complete Independence**:
   - Each question must be self-contained with ALL necessary information
   - If a question is incomplete, incorporate missing context to make it
   fully solvable
   - Multiple questions must be mutually independent (no "as in first
   Question ", "using previous result")
4. **Answer Must Exist**: All answers must be directly found or clearly
derivable from the given text - do not extract if uncertain

## For Questions/Problems:
- Extract any explicit physics questions ONLY if their answers are present in
the document
- If you see only solution steps without a question, DO NOT create a question
- **For Laws/Principles with proofs/derivations**: Create questions asking
for the derivation or proof when available
- **For Standalone Laws/Principles** (without derivation): Create questions
about their content, applications, or physical meaning ONLY if the answer is
in the text
- **For Formulas**: Create questions flexibly (e.g., if text derives E=mc²,
could ask "Derive the mass-energy relation" or "What is the relationship
between mass and energy?" or apply it in a scenario) ONLY if the answer
content is in the text
- **For Concepts/Definitions**: Ask about them flexibly, but ensure the
complete answer can be found in the document
- **For Phenomena Explanations**: Create questions naturally based on the
content
- Extract multiple questions separately if they exist, ensuring each is
completely independent without relying on or referencing other questions
- If no valid physics Q&A content can be extracted following above rules,
return `[NO QA]`

## For Answers:
- **Process: First locate the answer in the document, then enhance the
reasoning steps**
```

```
- Include the provided solution, derivation, or explanation when available
- For derivations/proofs, the answer should contain the complete step-by-step
process
- If the answer already has explanation, reorganize it into a clear and
well-structured format for better readability and understanding
- If the answer lacks explanation, add necessary intermediate reasoning
(physical reasoning, mathematical steps, unit analysis, physical
interpretation)
- For final answers that need exact matching (multiple-choice, calculations,
fill-in-the-blank, true/false), use $
boxed$ notation

## Format:
Use this exact format for parsing:

---BEGIN QA---
Question: <First complete question>
Answer: <First complete answer>
---END QA---

---BEGIN QA---
Question: <Second complete question>
Answer: <Second complete answer>
---END QA---


The extract is as follows:
chunk

Now process the extract and return the result.
```

QA Extraction Prompt for Darwin-Science-Book-QA(stem$_{others}$)

```
Below is a document extract from STEM fields (engineering, applied sciences,
technology, interdisciplinary sciences, etc.).

# Extraction Task
Extract complete, independently solvable STEM Q&A pairs following these
strict guidelines:

## CRITICAL RULES (Must Follow):
1. **Answer-First Principle**: Only extract a question if its complete answer
exists in the document
2. **Zero References Requirement**:
   - Questions: No references to "the document", "the text", "Figure 1",
   "Table 2", "Equation 3", "above", "below", etc.
   - Answers: No references to "as in document", "as shown in text", "see
   Figure", "Equation 1", etc.
   - Replace ALL references with actual content from the document
3. **Complete Independence**:
   - Each question must be self-contained with ALL necessary information
   - If a question is incomplete, incorporate missing context to make it
   fully solvable
   - Multiple questions must be mutually independent (no "as in first
   Question", "using previous result")
4. **Answer Must Exist**: All answers must be directly found or clearly
derivable from the given text - do not extract if uncertain
5. **Value-Based Selection**: Focus on extracting the MOST IMPORTANT and
EDUCATIONAL content
7. **No Figure/Table-Dependent Content**: Do NOT extract questions whose
answers rely heavily on complex diagrams, charts, or data tables that cannot
be adequately described in text

## For Questions/Problems:
- **Extract explicit question-answer pairs** if they exist (ensure zero
references and independence requirements)
```

```
- **Prioritize substantive technical and scientific content** over trivial
details or data-dependent conclusions
- **Create questions about**:
  - Principles and theories: fundamental concepts, laws, theoretical
  frameworks, underlying principles
  - Formulas and equations: mathematical expressions, relationships,
  derivations (when describable in text)
  - Mechanisms and processes: how systems work, operational principles,
  step-by-step processes
  - Concepts and definitions: technical terminology, key concepts,
  classifications
  - Properties and characteristics: material properties, system
  characteristics, performance parameters
  - Comparisons: differences between systems/methods/materials/approaches
- **Critical**: Each question must be self-contained with all necessary
technical context, definitions, conditions, and parameters
- **Do NOT extract**:
  - Questions depending on complex diagrams, charts, or detailed data tables
  - Questions requiring extensive background knowledge not provided in the
  text
  - Trivial details without educational or technical significance
  - Questions whose answers are uncertain or incomplete
- If no valid Q&A content can be extracted following above rules, return
exactly: [NO QA]

## For Answers:

- **Process**: First locate the complete answer in the document, then
organize it clearly
- If the answer has explanation, reorganize it into a clear, well-structured
format
- If the answer lacks explanation, add necessary technical reasoning or
context
- For final answers that need exact matching (multiple-choice, calculations,
selections, specific values), use $
boxedanswer$ notation

## Output Format (STRICT):
Use this exact format for parsing:

---BEGIN QA---
Question: <First complete question>
Answer: <First complete answer>
---END QA---

---BEGIN QA---
Question: <Second complete question>
Answer: <Second complete answer>
---END QA---

If no valid Q&A can be extracted, return exactly: [NO QA]

---
Extract:
chunk

Now process the extract and return the result in the specified format.
```